

Swiss Finance Institute

Research Paper Series

N°26-20

Large and Deep Factor Models



Bryan Kelly

Yale School of Management, AQR Capital Management, and NBER

Boris Kuznetsov

AQR Capital Management

Semyon Malamud

Ecole Polytechnique Fédérale de Lausanne, Swiss Finance Institute, and CEPR

Teng Andrea Xu

AQR Capital Management

Yuan Zhang

Shanghai University of Finance and Economics

Large and Deep Factor Models

Bryan Kelly, Boris Kuznetsov, Semyon Malamud, Teng Andrea Xu,
and Yuan Zhang*

March 2, 2026

Abstract

We show that a deep neural network (DNN) trained to construct a stochastic discount factor (SDF) admits a sharp additive decomposition that separates nonlinear characteristic discovery from the pricing rule that aggregates them. The economically relevant component of this decomposition is governed by a new object, the Portfolio Tangent Kernel (PTK), which captures the features learned by the network and induces an explicit linear factor pricing representation for the SDF. In population, the PTK-implied SDF converges to a ridge-regularized version of the true SDF, with the effective strength of regularization determined by the spectral complexity of the PTK. Using U.S. equity data, we show that the PTK representation delivers large and statistically significant performance gains, while its spectral complexity has risen sharply—by roughly a factor of six since the early 2000s—imposing increasingly tight limits on finite-sample pricing performance.

Keywords: Asset pricing; stochastic discount factor; neural networks; factor models

JEL classification: G12; C45; C52

*Bryan Kelly is at Yale School of Management, AQR Capital Management, and NBER; www.bryankellyacademic.org. Boris Kuznetsov is at AQR Capital Management. Semyon Malamud is at the Swiss Finance Institute, EPFL, and CEPR, and is a consultant to AQR. Teng Andrea Xu is at AQR Capital Management. Yuan Zhang is at the Shanghai University of Finance and Economics. Semyon Malamud gratefully acknowledges the financial support of the Swiss Finance Institute and the Swiss National Science Foundation, Grant 100018_192692. AQR Capital Management is a global investment management firm that may or may not apply similar investment techniques or methods of analysis as described herein. The views expressed here are those of the authors and not necessarily those of AQR. This work was supported by a grant from the Swiss National Supercomputing Center (CSCS) under project ID lp46. We thank Mohammad Pourmohammadi for his excellent comments and suggestions.

1. Introduction

Modern asset pricing increasingly relies on high-dimensional data and flexible statistical methods. A growing literature shows that deep neural networks (DNNs) can construct stochastic discount factors (SDFs), return forecasts, and portfolio strategies that outperform traditional linear factor models. These empirical successes raise a fundamental but unresolved question: what, exactly, do neural networks learn when trained on financial data?

From an economic perspective, training a neural network implicitly performs two distinct tasks. First, the network learns *what to price*: nonlinear features constructed from firm characteristics that summarize economically relevant information. Second, it learns *how to price*: a rule that aggregates those features into an SDF. In standard implementations, these two tasks are learned jointly through gradient descent and are therefore tightly entangled. As a result, the network must simultaneously discover informative features and estimate their optimal combination from finite samples. This joint learning problem obscures economic interpretation and can lead to statistical inefficiencies, reinforcing the perception of neural networks as black boxes that are difficult to discipline using asset pricing theory.

This paper shows that DNN-based SDFs admit a sharp additive decomposition that separates feature learning from pricing. For networks trained by gradient descent, the estimated SDF can be written as the sum of two components: a feature-based term that depends only on the nonlinear characteristics learned by the network, and a residual model-dependent term. We show that the feature-based component admits a closed-form representation governed by a new object, the *Portfolio Tangent Kernel* (PTK).

Conditional on the learned characteristics, the PTK induces a unique linear pricing

rule. The resulting SDF—what we call the *PTK-SDF*—is the Markowitz portfolio of a very large collection of characteristic-managed factors. This representation takes the form of a Large Factor Model (LFM). Because this pricing rule is optimal conditional on the learned features, the residual model-dependent component of the original DNN does not improve performance and can be discarded. In this sense, neural networks are best understood as powerful devices for feature discovery, while the PTK provides a cleaner, transparent, and statistically efficient way to price those features.

Empirically, this distinction is decisive. Using U.S. equity data, we show that the PTK-SDF systematically outperforms the original DNN-SDF, delivering higher Sharpe ratios and economically significant alphas across asset size groups. Conditioning on learned features while re-optimizing the pricing rule mitigates learning-induced noise that arises when feature discovery and pricing are conducted jointly. This leads to a modular estimation pipeline: a neural network is used solely to learn nonlinear characteristics; these characteristics are mapped into a large set of characteristic-based factors; and the factors are combined using an explicit, regularized portfolio rule. The PTK-SDF therefore isolates—and improves upon—the economically relevant content of the original network.

The closed-form PTK representation also allows us to characterize the statistical limits of DNN-based asset pricing models. Because the number of learned factors is extremely large relative to available time-series observations, classical inference breaks down. Building on recent results in random matrix theory, we show that performance is governed by two forces. The first is *alignment*: the extent to which risk premia load on statistically strong directions of factor risk. The second is *spectral complexity*: the dispersion of factor risk across principal components, which determines the effective degree of regularization faced by the investor. Higher spectral complexity corresponds to a flatter risk spectrum, in which economically relevant variation is spread across

many weak directions. In finite samples, this dispersion amplifies estimation error and limits achievable Sharpe ratios.

We document substantial time variation in the spectral complexity of the PTK. Over the past two decades, it has increased by roughly a factor of six, indicating a pronounced rise in the statistical complexity of the SDF. This increase coincides with the well-documented decline in factor model performance since the early 2000s. At the same time, alignment between risk premia and factor risk has improved, partially offsetting the adverse effects of rising complexity. These findings highlight a fundamental trade-off: feature learning improves economic alignment, but rising spectral complexity imposes limits on what can be learned from finite samples.

An important implication of this framework is that feature learning fundamentally reshapes regularization. We find that factors constructed from learned features exhibit substantially higher spectral complexity than factors constructed from random features, such as those studied by [Didisheim et al. \(2024\)](#). Quantitatively, the spectral complexity of learned-feature-based factors is roughly an order of magnitude larger. As a result, these factors are subject to strong *implicit* regularization even in the absence of explicit ridge penalties. In particular, the PTK-SDF achieves its best performance with little or no additional shrinkage: despite involving hundreds of thousands of factors, the optimal pricing rule is effectively ridgeless.¹

Beyond statistical performance, the PTK representation restores economic discipline. We show that PTK-based SDFs exhibit substantially stronger alignment with long-horizon and future consumption risk—both in the sense of [Parker and Julliard \(2005\)](#) and its micro-founded extension in [Malamud et al. \(2025\)](#)—than either fully trained DNN-SDFs or random-feature benchmarks.

¹Here, “ridgeless” does not mean unregularized. The PTK-SDF remains strongly disciplined through the spectral complexity of the learned factors, which endogenously shrinks weak risk directions even in the absence of an explicit penalty.

Taken together, our findings provide a unified framework for understanding deep learning in asset pricing. Neural networks learn economically informative features, but pricing those features efficiently requires separating feature discovery from model estimation. The Portfolio Tangent Kernel performs this separation by replacing the original DNN with an explicit large-factor pricing representation. This representation allows both the economic content of learned features and the statistical limits imposed by finite samples to be analyzed using modern asset pricing tools rooted in high-dimensional statistics and random matrix theory.

2. Related Literature

A central theme in asset pricing is the tension between sparsity and complexity in the construction of stochastic discount factors (SDFs). While classical factor models emphasize parsimony, a large literature documents that richly parameterized models substantially outperform sparse specifications out of sample (e.g., [Cochrane, 2011](#); [Harvey et al., 2016](#); [McLean and Pontiff, 2016](#); [Kelly et al., 2020](#); [Gu et al., 2020a](#); [Hou et al., 2020](#); [Didisheim et al., 2024](#)). See ([Kelly and Xiu, 2023](#)) for an overview.

These findings motivate Large Factor Models (LFMs), which construct SDFs from extensive collections of characteristic-based factors. With appropriate shrinkage, LFMs perform well even when based on simple linear characteristics ([Kozak et al., 2020](#); [Kelly et al., 2023](#)). At the same time, approximating the true conditional SDF may require an extremely large number of factors. ([Didisheim et al., 2024](#)) show that, in the limit of infinitely many factors, the unconditionally optimal factor portfolio converges to the true conditional SDF, despite the latter being infeasible to estimate directly.

The use of highly over-parameterized factor models raises natural statistical con-

cerns. Estimation error becomes severe when the number of factors exceeds the sample size, and classical inference breaks down. Nevertheless, (Didisheim et al., 2024), building on the virtue-of-complexity framework of (Kelly et al., 2022), show that extreme dimensionality can improve out-of-sample performance. A key insight is that regularization (in the form of effective shrinkage) arises endogenously. (Didisheim et al., 2024), (Kelly and Malamud, 2025), and (Chernov et al., 2025) show that effective shrinkage is determined by the spectral properties of covariance matrices, while (Chernov et al., 2025) emphasize the role of alignment between factor risk premia and factor risk in determining achievable Sharpe ratios.

These insights motivate new tools for inference. When the number of factors exceeds the sample size, the classical (Gibbons et al., 1989) statistic diverges. (Chernov et al., 2025) propose a debiased GRS statistic that converges to a ridge-penalized population Sharpe ratio and provides a direct measure of alignment. We adopt this statistic in our empirical analysis.

Most existing Large Factor Models rely on linear factors or random nonlinear transformations. In particular, the model studied by (Didisheim et al., 2024) is equivalent to a shallow but wide neural network with untrained hidden-layer weights. In contrast, much of the machine-learning-based asset pricing literature employs fully trained, multi-layer networks whose economic content and statistical properties are difficult to analyze theoretically (e.g., Chen et al., 2024; Gu et al., 2020b; Fan et al., 2022).

This paper bridges these strands by distinguishing between feature learning and pricing in deep learning-based asset pricing models. We study fully trained neural networks of arbitrary depth and width that remain analytically tractable using Neural Tangent Kernel methods (Jacot et al., 2018; Yang, 2020). We show that, while the DNN learns economically informative nonlinear features, the associated pricing rule

need not be optimal. Instead, the learned features admit a closed-form Large Factor Model representation governed by a novel kernel, the Portfolio Tangent Kernel (PTK), which delivers an optimal pricing rule conditional on those features. This perspective clarifies how feature learning reshapes spectral complexity, implicit regularization, and alignment, and connects deep learning models to the emerging theory of high-dimensional asset pricing.

3. Parametric SDFs, Kernels, and Neural Networks

We consider a panel of stocks indexed by $i = 1, \dots, N_t$, with excess returns

$$R_{t+1} = (R_{i,t+1})_{i=1}^{N_t}$$

observed at time $t + 1$. Each stock i is associated with a d -dimensional vector of characteristics

$$X_{i,t} = (X_{i,t}(k))_{k=1}^d,$$

and we collect all characteristics in the matrix

$$X_t = [X_t(1), \dots, X_t(d)] \in \mathbb{R}^{N_t \times d}.$$

The true tradable stochastic discount factor (SDF) is given by

$$M_{t+1} = 1 - \pi_t' R_{t+1}, \tag{1}$$

where π_t denotes the conditionally efficient portfolio

$$\pi_t = E_t[R_{t+1}R'_{t+1}]^{-1} E_t[R_{t+1}], \quad (2)$$

which solves the utility maximization problem

$$\max_{\pi_t} E_t[\pi'_t R_{t+1} - 0.5(\pi'_t R_{t+1})^2]. \quad (3)$$

If X_t contains all relevant conditioning information, then π_t in (2) can be written as

$$\pi_t = \pi(X_t), \quad (4)$$

for some unknown and potentially highly nonlinear function $\pi(X)$. Using the law of iterated expectations, the problem of finding the conditionally optimal portfolio policy can then be rewritten as an *unconditional*, non-parametric utility maximization problem:

$$\min_{\pi(\cdot)} E[(1 - \pi(X_t)'R_{t+1})^2] = 1 - 2 \max_{\pi(\cdot)} E[U(\pi(X_t)'R_{t+1})], \quad (5)$$

where $U(x) = x - 0.5x^2$. A standard approach to solving (5) for $\pi(X)$ is to posit a sufficiently rich parametric family of functions $f(x; \theta)$ and to estimate the parameter vector θ using sample analogs of the objective in (5). Following (Didisheim et al., 2024), we consider a ridge-penalized version of this objective,

$$\min_{\theta} L(\theta), \text{ where } L(\theta) = \left(\frac{1}{T} \sum_{t=1}^T (1 - f(X_t; \theta)'R_{t+1})^2 + z \|\theta\|^2 \right), \quad (6)$$

where z denotes the ridge penalty and serves as a regularization parameter.

Equation (6) highlights the central tension underlying high-dimensional SDF estimation. Richer function families reduce approximation bias by allowing rich functional forms for $\pi(X)$, but they simultaneously exacerbate estimation error when the dimensionality of θ is large relative to the sample size. As a result, improvements in in-sample fit need not translate into better out-of-sample SDF performance. Our contribution is to characterize how this bias–variance tradeoff manifests itself in MSRR-based SDF estimation and to show that, in finite samples, limits to learning impose sharp constraints on the achievable Sharpe ratio, even when the true SDF lies within the span of the chosen function family.

A further complication arises from computation. When θ is high-dimensional, solving (6) exactly is generally infeasible. In practice, estimation relies on iterative optimization algorithms that converge to particular local minima. Recent work in machine learning emphasizes that the choice of optimization algorithm plays a central role in determining out-of-sample performance; see, for example, (Zhou et al., 2023). Thus, not only the function family $f(x; \theta)$, but also the manner in which (6) is solved, matters for the properties of the resulting SDF.

The empirical success of deep learning reflects the fact that certain function families—deep neural networks combined with gradient-based optimization—can achieve good generalization even in regimes where the number of parameters exceeds the available sample size; see, for example, (Nakkiran et al., 2021) and (Belkin, 2021). For asset pricing, this raises two distinct questions: what features of the data are learned by these models, and how those features are combined into an SDF. In the sections that follow, we show that addressing these questions separately is key to understanding both the empirical performance and the statistical limits of neural-network-based SDFs.

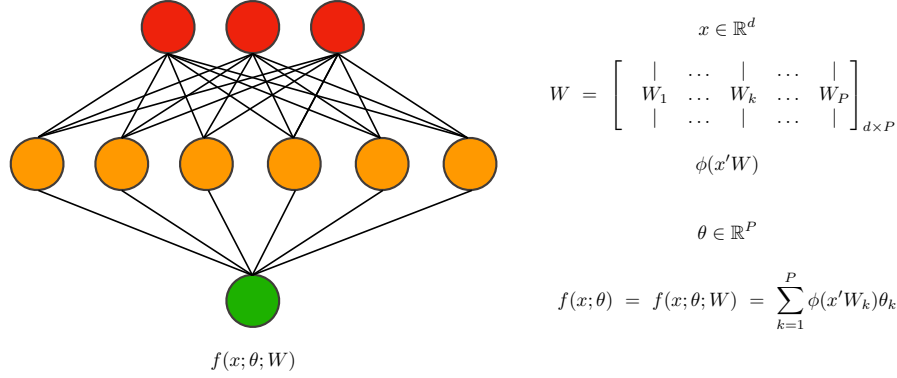


Figure 1: The figure above shows $f(x; \theta; W)$, a mathematical representation of a neural network with a single hidden layer, also known as a shallow network. The weights $W \in \mathbb{R}^{d \times P}$ and $\theta \in \mathbb{R}^P$ are randomly initialized. $\phi : \mathbb{R} \rightarrow \mathbb{R}$ is an elementwise non-linear activation function and the vector $\phi(x'W)$ is referred to as *random features*.

3.1. Random Features and Large Factor Models

Before turning to fully trained deep neural networks, it is useful to consider simpler function families that depend linearly on the parameter vector $\theta \in \mathbb{R}^P$. These models provide a transparent benchmark in which features are fixed rather than learned, allowing for an explicit characterization of pricing, regularization, and asymptotic behavior. Following (Didisheim et al., 2024), we consider the family

$$f(x; \theta) = f(x; \theta; W) = \sum_{k=1}^P \phi(x'W_k) \theta_k, \quad (7)$$

where $x \in \mathbb{R}^d$ denotes the vector of raw characteristics, $W_k \in \mathbb{R}^d$ are randomly drawn weights, and $\phi(\cdot)$ is a nonlinear activation function. The nonlinear functions $\phi(x'W_k)$ are referred to as random features. As explained in (Didisheim et al., 2024), the

specification in (7) is equivalent to a shallow, single-hidden-layer neural network in which the output-layer weights θ_k are estimated, while the hidden-layer weights W_k are held fixed and not optimized. Figure 1 provides a schematic illustration.

The linearity of the function family in (7) implies that the optimization problem in (6) admits a unique closed-form solution. Moreover, this solution can be expressed in terms of characteristics-managed portfolios, or factors. Specifically, defining the vector of factor returns $F_{t+1} = (F_{k,t+1})_{k=1}^P$ with

$$F_{k,t+1} = \sum_{i=1}^{N_t} \underbrace{\phi(X'_{i,t}, W_k)}_{k\text{-th random feature}} R_{i,t+1}, \quad (8)$$

we can rewrite the SDF as a factor portfolio,

$$\sum_{i=1}^{N_t} f(X_{i,t}; \theta) R_{i,t+1} = \theta' F_{t+1}. \quad (9)$$

Substituting (9) into (6) yields

$$\theta(z) = \arg \min_{\theta} L(\theta) = \left(zI + T^{-1} \sum_{t=1}^T F_t F_t' \right)^{-1} T^{-1} \sum_{t=1}^T F_t. \quad (10)$$

Thus, within the linear family (7), the estimated SDF admits a clear and familiar interpretation as the mean–variance efficient portfolio of a (potentially very large) collection of factors.

Standard intuition based on the Arbitrage Pricing Theory of (Ross, 1976) suggests that the number of factors required to span the SDF should be small. However, (Didisheim et al., 2024) show both theoretically and empirically that this intuition fails in high-dimensional settings. They establish the *virtue of complexity*, whereby out-of-sample performance is monotonically increasing in the number of factors P ,

even in the limit as $P \rightarrow \infty$. This result naturally raises the question of the limiting behavior of the estimated SDF as the number of random features grows. Addressing this question requires introducing tools from kernel theory.

A positive definite kernel on $\mathbb{R}^D$ is a function $K(x, \tilde{x}) : \mathbb{R}^D \times \mathbb{R}^D \rightarrow \mathbb{R}$ such that, for any collection of vectors $x_i \in \mathbb{R}^D, i = 1, \dots, n$, and any coefficient vector $\alpha \in \mathbb{R}^n$,

$$\sum_{i_1=1}^n \sum_{i_2=1}^n \alpha_{i_1} \alpha_{i_2} K(x_{i_1}, x_{i_2}) \geq 0. \quad (11)$$

Equivalently, the kernel matrix $\bar{K} = (K(x_{i_1}, x_{i_2}))_{i_1, i_2=1}^n$ is positive semidefinite. Given a collection of nonlinear functions $\phi_k(x), k = 1, \dots, P$, commonly referred to as features, one can define the associated kernel

$$K(x, \tilde{x}) = \frac{1}{P} \sum_{k=1}^P \phi_k(x) \phi_k(\tilde{x}). \quad (12)$$

By direct calculation, K is positive semidefinite.² Conversely, any positive definite kernel can be approximated by a sufficiently rich collection of nonlinear features: By Mercer's theorem (see, e.g., [Bartlett, 2003](#); [Rahimi and Recht, 2007](#)), any positive definite kernel $K(x, \tilde{x})$ admits the representation

$$K(x, \tilde{x}) = \int \phi(x; \omega) \phi(\tilde{x}; \omega) p(d\omega), \quad (13)$$

for some probability distribution $p(d\omega)$. Sampling W_k from $p(d\omega)$ yields the Monte

²Indeed, $\sum_{i_1=1}^n \sum_{i_2=1}^n \alpha_{i_1} \alpha_{i_2} K(x_{i_1}, x_{i_2}) = \frac{1}{P} \sum_k (\sum_i \phi_k(x_i) \alpha_i)^2 \geq 0$.

Carlo approximation

$$K(x, \tilde{x}) = \lim_{P \rightarrow \infty} \frac{1}{P} \sum_{k=1}^P \phi(x; W_k) \phi(\tilde{x}; W_k). \quad (14)$$

This representation implies that, as $P \rightarrow \infty$, the SDF based on the optimal θ in (10) converges to a well-defined limit determined solely by the kernel K . To characterize this limit in an asset-pricing setting, we introduce the following definition.

Definition 1 (The Portfolio Kernel) *Let*

$$Y_t = (R_{t+1}, X_t) \in \mathbb{R}^{N_t(d+1)} \quad (15)$$

denote the market state, encompassing all stock characteristics and all stock returns. Given any two market states $Y = (R, X)$, $\tilde{Y} = (\tilde{R}, \tilde{X})$, with $R \in \mathbb{R}^N$, $\tilde{R} \in \mathbb{R}^{\tilde{N}}$, we let

$$K(X, \tilde{X}) = (K(X_i, \tilde{X}_j)) \in \mathbb{R}^{N \times \tilde{N}}, \quad (16)$$

and define the Portfolio Kernel as

$$\mathbb{K}(Y; \tilde{Y}) = \underbrace{R'}_{1 \times N} K(X, \tilde{X}) \underbrace{\tilde{R}}_{\tilde{N} \times 1}. \quad (17)$$

The Portfolio Kernel operates on market states and produces a kernel-based similarity measure $\mathbb{K}(Y; \tilde{Y})$. Importantly, this similarity measure is itself a portfolio return, as it aggregates asset returns using weights determined by the similarity of characteristics across market states.

We denote the in-sample data by

$$Y_{IS} = (R_{t+1}, X_t)_{t=1}^T \in \mathbb{R}^{T \times N_t(d+1)}, \quad (18)$$

and define the corresponding in-sample kernel matrix

$$\mathbb{K}_{IS} = (\mathbb{K}(Y_{t_1}; Y_{t_2}))_{t_1, t_2=1}^T \in \mathbb{R}^{T \times T}, \quad (19)$$

which measures similarity across in-sample market states.

Let K be a positive definite kernel admitting the representation (13). Let $\{W_k\}_{k=1}^P$ be independently sampled from $p(d\omega)$ in (13), and define the factor vector $F_{t+1} = (F_{k,t+1})_{k=1}^P$ by

$$F_{k,t+1} = \frac{1}{P^{1/2}} \sum_{i=1}^{N_t} \phi(X_{i,t}; W_k) R_{i,t+1}. \quad (20)$$

Equivalently,

$$F_{t+1} = \frac{1}{P^{1/2}} \phi(X_t; W) R_{t+1} \in \mathbb{R}^P. \quad (21)$$

Let

$$\theta(z; T) = \left(zI + T^{-1} \sum_{t=1}^T F_t F_t' \right)^{-1} T^{-1} \sum_{t=1}^T F_t \quad (22)$$

denote the ridge-penalized efficient factor portfolio, and let $\theta' F_{t+1}$ be the correspond-

ing SDF. By direct calculation,

$$\begin{aligned}
F'_{t+1}F_{\tau+1} &= \frac{1}{P}R'_{t+1}\phi(X_t;W)\phi(X_\tau;W)'R_{\tau+1} \\
&= R'_{t+1}\left(\frac{1}{P}\sum_{k=1}^P\phi(X_t;W_k)\phi(X_\tau;W_k)'\right)R_{\tau+1} \\
&\stackrel{(14)}{\rightarrow} R'_{t+1}K(X_t,X_\tau)R_{\tau+1} = \mathbb{K}(Y_t,Y_\tau).
\end{aligned} \tag{23}$$

Denoting by $F = (F_t)_{t=1}^T \in \mathbb{R}^{P \times T}$ the matrix of in-sample factor returns, it follows that

$$F'F = (F'_{t_1}F_{t_2})_{t_1,t_2=1}^T \approx (\mathbb{K}(Y_{t_1},Y_{t_2}))_{t_1,t_2=1}^T = \mathbb{K}_{IS}. \tag{24}$$

We will require the following technical lemma.

Lemma 1 *Let $F \in \mathbb{R}^{P \times T}$ denote the matrix of in-sample factor returns, and let $\mathbf{1} = (1, \dots, 1) \in \mathbb{R}^T$ be the vector of ones. Then,*

$$\theta = T^{-1}(zI + T^{-1}FF')^{-1}F\mathbf{1} = T^{-1}F(zI + T^{-1}F'F)^{-1}\mathbf{1}. \tag{25}$$

Lemma 1 follows directly from the regression objective (6): θ is the coefficient vector obtained by regressing $\mathbf{1}$ on factor returns. Consequently,

$$F'_{T+1}\theta = T^{-1} \underbrace{F'_{T+1}F}_{\rightarrow \mathbb{K}(Y_{T+1},Y_{IS})} \left(zI + T^{-1} \underbrace{F'F}_{\rightarrow \mathbb{K}_{IS}} \right)^{-1} \mathbf{1}, \tag{26}$$

and we obtain the following result.

Theorem 1 (LFM-SDF) *The Large Factor Model (LFM) SDF is given by*

$$R^{LFM}(z)_{T+1} = \lim_{P \rightarrow \infty} \theta' F_{T+1} = \mathbb{K}(Y_{T+1}, Y_{IS}) \xi = \sum_{t=1}^T \underbrace{\mathbb{K}(Y_{T+1}, Y_t)}_{\text{attention to } t} \underbrace{\xi_t}_{\text{optimal weight}}, \quad (27)$$

where

$$\xi = \frac{1}{T} (zI + T^{-1} \mathbb{K}_{IS})^{-1} \mathbf{1}. \quad (28)$$

Theorem 1 characterizes the asymptotic behavior of the random-feature-based Large Factor Model studied in (Didisheim et al., 2024). In the limit of infinitely many random features, the estimated SDF converges to a well-defined object governed entirely by the kernel K .

The LFM-SDF compares the current market state Y_{T+1} with past realizations Y_t , $t = 1, \dots, T$, using the Portfolio Kernel $\mathbb{K}$. Periods that are more similar to the present receive higher weight, while less similar periods receive lower weight. These similarity scores are then aggregated in a mean-variance efficient manner through the weight vector $\xi \in \mathbb{R}^T$, which can be interpreted as optimal time fixed effects.

Although the kernel representation emphasizes aggregation over time, the LFM-SDF also admits a conventional portfolio representation in terms of characteristics-managed factors.

Corollary 2 *We have*

$$\lim_{P \rightarrow \infty} \theta' F_{T+1} = R'_{T+1} f^{\mathbb{K}}(X_t), \quad (29)$$

where

$$f^{\mathbb{K}}(x) = \sum_{t=1}^T \left(\sum_{j=1}^{N_t} R_{j,t+1} K(x, X_{j,t}) \right) \xi_t. \quad (30)$$

The representation of $f^{\mathbb{K}}(x)$ provides a transparent interpretation of how the LFM extracts pricing information from the joint distribution of returns and characteristics. For any out-of-sample characteristic vector x , the model evaluates its similarity to past characteristics through the kernel $K(x, X_{j,t})$ and forms the similarity-weighted performance measure

$$Perf_t(x) = \sum_{j=1}^{N_t} R_{j,t+1} K(x, X_{j,t}). \quad (31)$$

The final portfolio weight is obtained by optimally aggregating these performance measures across time,

$$f^{\mathbb{K}}(x) = \sum_{t=1}^T Perf_t(x) \xi_t. \quad (32)$$

In summary, once a kernel K is specified, the analysis delivers an explicit closed-form SDF that efficiently aggregates an effectively infinite number of characteristics-based factors. This property motivates the term Large Factor Model. The analysis also highlights a limitation: the kernel K is exogenously chosen rather than learned. In the next section, we show how fully trained neural networks endogenously learn such kernels through gradient-based optimization, leading to the Portfolio Tangent Kernel.

4. The Portfolio Tangent Kernel

We consider a parametric function family $f(X; \theta) : \mathbb{R}^{N \times d} \rightarrow \mathbb{R}^N$ mapping characteristics $X \in \mathbb{R}^{N \times d}$ to portfolio weights. For example, an own-characteristics neural network $g(X_{i,t}; \theta)$ studied in [Gu et al. \(2020b\)](#); [Chen et al. \(2024\)](#) implies $f(X_t; \theta) = (g(X_{i,t}; \theta))_{i=1}^N$, while a transformer architecture as in [Kelly et al. \(2025\)](#) yields a mapping that cannot be decoupled across stocks because each coordinate depends on the characteristics of all assets. Throughout, we refer to the function family as a DNN, although it may originate from other families, such as panel trees ([Cong et al., 2025](#)).

4.1. Optimization, Initialization, and Gradient Flow

In practice, DNNs are trained by randomly initializing the parameter vector $\theta = \theta(0)$ and then updating it using gradient-based optimization. This initialization is economically relevant: in over-parameterized settings, many parameter vectors achieve identical in-sample fit, so the learned SDF depends not only on the objective function but also on the optimization path induced by the algorithm.

Letting $\eta > 0$ denote the learning rate, gradient descent updates the parameters according to

$$\theta(s + ds) = \theta(s) - \eta ds \nabla_{\theta} L(\theta(s)), \quad (33)$$

where $s \geq 0$ indexes (rescaled) training time, or equivalently, the cumulative number of gradient descent steps. In the limit of an infinitesimal step size, this recursion

converges to the gradient flow

$$\frac{d\theta(s)}{ds} = -\eta \nabla_{\theta} L(\theta(s)). \quad (34)$$

The path $\{\theta(s)\}_{s \geq 0}$ therefore defines a sequence of candidate pricing rules generated by the learning algorithm.

While the dynamics of the high-dimensional parameter vector $\theta(s)$ are generally intractable, the evolution of the portfolio return generated by the network admits a simpler characterization. Applying the chain rule yields

$$\frac{d}{ds}(R'f(x; \theta(s))) = -\eta R' \nabla_{\theta} f(x; \theta(s)) \nabla_{\theta} L(\theta(s)). \quad (35)$$

Equation (35) shows that gradient descent does not update the SDF directly. Instead, learning operates through the gradient feature portfolio returns $R' \nabla_{\theta} f(x; \theta(s))$, which determine how pricing errors translate into changes in portfolio weights. From an asset pricing perspective, these gradient feature portfolios define a large collection of characteristics-based payoffs that are implicitly traded during training. This observation motivates a kernel representation that measures similarity across market states through the interaction of returns with gradient features.

4.2. The Portfolio Tangent Kernel

Definition 2 (The Portfolio Tangent Kernel (PTK)) *Let $R \in \mathbb{R}^N$, $\tilde{R} \in \mathbb{R}^{\tilde{N}}$ be two return vectors, and let $X \in \mathbb{R}^{N \times P}$, $\tilde{X} \in \mathbb{R}^{\tilde{N} \times P}$ denote their characteristics. Define*

$$\mathbb{K}((R, X); (\tilde{R}, \tilde{X}); \theta) = \underbrace{R'}_{1 \times N} \underbrace{\nabla_{\theta} f(X; \theta)}_{N \times P} \underbrace{\nabla_{\theta} f(\tilde{X}; \theta)'}_{P \times \tilde{N}} \underbrace{\tilde{R}}_{\tilde{N} \times 1} \quad (36)$$

to be the tangent kernel associated with the portfolio mapping $f(X; \theta)$. We refer to this object as the *Portfolio Tangent Kernel (PTK)*.

The PTK is a pricing-relevant kernel defined on market states. It measures similarity between states by aggregating returns using weights determined by the sensitivity of portfolio payoffs to parameter perturbations. By construction, the PTK is a Portfolio Kernel in the sense of Definition 1, with the underlying kernel given by the Neural Tangent Kernel (NTK; Jacot et al., 2018),³

$$K(X, \tilde{X}; \theta) = \nabla_{\theta} f(X; \theta) \nabla_{\theta} f(\tilde{X}; \theta)'. \quad (37)$$

Accordingly,

$$\mathbb{K}((R, X); (\tilde{R}, \tilde{X}); \theta) = R' K(X, \tilde{X}; \theta) \tilde{R}. \quad (38)$$

While the NTK captures similarity across characteristics, the PTK lifts this structure to traded returns and therefore governs pricing dynamics rather than prediction alone.

When θ is high-dimensional, the PTK aggregates a large collection of characteristics-managed factors,

$$F_{t+1} = (F_{k,t+1})_{k=1}^P, \quad F_{k,t+1} = R'_{t+1} \nabla_{\theta_k} f(X_t; \theta). \quad (39)$$

For any two market states Y_t, Y_{τ} ,

$$\mathbb{K}(Y_t; Y_{\tau}; \theta) = F'_{t+1} F_{\tau+1}, \quad (40)$$

³When the NTK is computed using the DNN after training, it is commonly referred to as the after-kernel. See, Long (2021); Wei et al. (2022); Schwab et al. (2025).

so that the PTK measures similarity across time in terms of the realized payoffs of these learned factors.

4.3. Feature Learning and PTK

Equation (35) implies that, under gradient flow, the portfolio return generated by the DNN evolves according to

$$\frac{d}{ds}(R'f(X;\theta(s))) = -\eta F'_{t+1} \nabla_{\theta} L(\theta(s)). \quad (41)$$

Specializing to the MSRR objective yields the following result.

Theorem 3 (MSRR Gradient Flow and the PTK) *Suppose that*

$$L(\theta) = \frac{1}{T} \sum_{t=1}^T (1 - f(X_t; \theta)' R_{t+1})^2. \quad (42)$$

Under gradient flow,

$$\frac{d}{ds}(R'f(X;\theta(s))) = -\eta T^{-1} \underbrace{\mathbb{K}(Y; Y_{IS}; \theta(s))}_{PTK} \underbrace{(\mathbf{1}_T - R'_{IS} f(X_{IS}; \theta(s)))}_{in-sample \text{ error}}, \quad (43)$$

where $Y_{IS} = (R_{IS}, X_{IS})$ is in-sample (IS) data, and

$$\mathbb{K}(Y; Y_{IS}; \theta) = (\mathbb{K}(Y; Y_t; \theta))_{t=1}^T \in \mathbb{R}^{1 \times T}. \quad (44)$$

Theorem 3 shows that gradient descent updates portfolio returns by comparing the current market state to past in-sample states through the PTK. Learning therefore operates through two distinct channels:

- *Model learning*, captured by the evolution of $f(X; \theta(s))$, which determines the portfolio weight function; and
- *Feature learning*, captured by the evolution of the PTK $\mathbb{K}(Y; Y_{IS}; \theta(s))$, reflecting learning of the gradient features $\nabla_{\theta} f(X; \theta)$ used to construct the traded factors.

After sufficiently many gradient descent steps, these dynamics stabilize. This occurs, for example, when $\theta(s)$ approaches a stationary point or when the network is sufficiently wide (Jacot et al., 2018). Once feature learning is complete, the PTK ceases to evolve, and the dynamics admit a closed-form solution.⁴

Theorem 4 (Closed-Form Stable-PTK DNN-SDFs) *Fix a large s_* and define*

$$\mathbb{K}^* = \mathbb{K}(\cdot; \cdot; \theta(s_*)). \quad (45)$$

Suppose that, after s_ gradient descent steps, the PTK stabilizes in the sense that*

$$\|\mathbb{K}(Y; Y_{IS}; \theta(s)) - \mathbb{K}^*(Y; Y_{IS})\| + \|\mathbb{K}(Y_{IS}; Y_{IS}; \theta(s)) - \mathbb{K}^*(Y_{IS}; Y_{IS})\| < \varepsilon \quad (46)$$

for all $s > s_$. Suppose further that*

$$\mathbb{K}_{IS}^* \equiv \mathbb{K}^*(Y_{IS}; Y_{IS}) \quad (47)$$

⁴The proof of this result follows directly from Theorem 3.

is non-degenerate. Then, for all $s > s_*$,

$$R'f(X; \theta(s)) = \underbrace{R'f(X; \theta(s_*))}_{\text{learned model}} + \underbrace{\frac{1}{T} \mathbb{K}^*(Y, Y_{IS}) \xi(s)}_{\text{learned features}} + O(\varepsilon), \quad (48)$$

where

$$\xi(s) = \left(\frac{1}{T} \mathbb{K}_{IS}^* \right)^{-1} \left(I - e^{-\eta(s-s_*) \frac{1}{T} \mathbb{K}_{IS}^*} \right) (\mathbf{1} - R'_{IS} f(X_{IS}, \theta(s_*))). \quad (49)$$

Theorem 4 delivers the central decomposition of the paper. Once the PTK has stabilized, gradient descent no longer alters the feature representation learned by the network. Instead, subsequent learning acts only through the aggregation of a fixed collection of gradient-based factors. In this regime, the DNN supplies the features, while pricing is governed entirely by the PTK.

This result makes precise the separation between *what is learned* and *how it is priced*. The term labeled *learned model* captures the residual pricing rule associated with the network at the stabilization point s_* . The second term—the PTK-SDF—corresponds to an explicit factor pricing model that optimally aggregates the learned features. Because this aggregation is efficient conditional on the features, the learned-model component does not improve pricing performance and can be discarded without loss.

The structure of $\xi(s)$ reveals that the PTK-SDF implements spectral shrinkage closely related to ridge regression. To see this, consider the spectral decomposition of the in-sample kernel matrix, $\mathbb{K}_{IS}^* = UDU'$, $D = \text{diag}(\lambda_i)_{i=1}^T$. Then,

$$\begin{aligned} \left(\frac{1}{T} \mathbb{K}_{IS}^* \right)^{-1} \left(I - e^{-\frac{1}{T} \eta \mathbb{K}_{IS}^* (s-s_*)} \right) &= U f_{GD}(D) U', \\ (zI + \frac{1}{T} \mathbb{K}_{IS}^*)^{-1} &= U f_{ridge}(D) U', \end{aligned} \quad (50)$$

where

$$\begin{aligned} f_{GD}(\lambda) &= \lambda^{-1} (1 - e^{-\eta(s-s_*)\lambda}), \\ f_{ridge}(\lambda) &= \lambda^{-1} \frac{1}{1 + \lambda^{-1}z}, \end{aligned} \tag{51}$$

are the spectral filters induced by gradient descent (GD) and ridge regularization, respectively. They both regularize the factor covariance matrix $\mathbb{K}_{IS}^*$ by downweighting directions associated with small eigenvalues. The key distinction is that, under gradient descent, shrinkage is generated endogenously by the learning dynamics rather than imposed ex ante through an explicit penalty. The effect of this shrinkage is most pronounced for small eigenvalues λ , for which

$$\begin{aligned} f_{GD}(\lambda) &= \eta(s - s_*) + O(\lambda), \\ f_{ridge}(\lambda) &= z^{-1} + O(\lambda). \end{aligned} \tag{52}$$

Thus, gradient descent induces an *effective* ridge penalty of order $1/(\eta(s - s_*))$.

If the kernel matrix $\mathbb{K}_{IS}^*$ is strictly positive definite, then as $s \rightarrow \infty$, the exponential term vanishes, and the PTK-SDF converges to the ridgeless kernel regression

$$\mathbb{K}^*(Y, Y_{IS}) (\mathbb{K}_{IS}^*)^{-1} \mathbf{1}. \tag{53}$$

As we show in the next section, although this estimator interpolates the in-sample data, it remains disciplined out of sample due to implicit spectral shrinkage induced by the PTK. The strength of this implicit regularization is governed by the spectral complexity of the PTK and plays a central role in shaping out-of-sample SDF performance.

5. Alignment and the Projection Theorem

Recent work emphasizes that limits to learning—the gap between the population-optimal SDF and the model learned from finite samples (Chen et al., 2025)—are governed by two fundamental properties of high-dimensional pricing models (Didisheim et al., 2024; Kelly and Malamud, 2025). The first is the spectral distribution of factor risk, which determines the severity of statistical shrinkage. The second is *alignment*: the extent to which economically relevant risk premia load on statistically strong directions of the factor space.

Random features are generically poorly aligned with the true SDF. Learned features, by contrast, may become substantially better aligned through training, allowing neural networks to outperform despite operating in extremely high dimensions.

We formalize these ideas using tools from Random Matrix Theory (RMT). Let $F \in \mathbb{R}^{T \times P}$ denote the matrix of factor returns, and define

$$\bar{F} = \frac{1}{T} \sum_{t=1}^T F_t, \quad \hat{\Sigma} = \frac{1}{T} \sum_{t=1}^T F_t F_t' - \bar{F} \bar{F}'.$$

Define the effective penalty

$$\hat{Z}_*(z) = \frac{1}{T^{-1} \operatorname{tr}((zI + FF'/T)^{-1})}. \quad (54)$$

Theorem 5 (Didisheim et al. (2024); Chernov et al. (2025)) *Suppose $F_{t+1} = \mu_P + \Sigma_P^{1/2} X_t$, where X_t has i.i.d. components with bounded fourth moments. Then,*

$$(zI + \hat{\Sigma})^{-1} \bar{F} - (\hat{Z}_*(z)I + \Sigma_P)^{-1} \mu_P \rightarrow 0 \quad (55)$$

in the weak sense as $P, T \rightarrow \infty$ with $P/T \rightarrow c > 0$.

Theorem 5 implies that all estimation noise collapses asymptotically to the scalar $\hat{Z}_*(z)$. When $P > T$, $\hat{Z}_*(0) > 0$, implying nontrivial shrinkage even for ridgeless estimators.

Let $\Sigma = UDU'$ be the population eigenvalue decomposition. Then

$$(\hat{Z}_*(z)I + \Sigma)^{-1}\mu = \sum_{i=1}^P \frac{U_i'\mu}{\lambda_i} \frac{1}{1 + \hat{Z}_*(z)\lambda_i^{-1}} U_i.$$

Theorem 6 (Projection Theorem) *If $\hat{Z}_*(z)$ separates the spectrum so that $\hat{Z}_*(z)/\lambda_i = O(\varepsilon)$ for $i \leq I$ and $\lambda_i/\hat{Z}_*(z) = O(\varepsilon)$ for $i > I$, then*

$$(\hat{Z}_*(z)I + \Sigma)^{-1}\mu = \sum_{i=1}^I \frac{U_i'\mu}{\lambda_i} U_i + O(\varepsilon).$$

The economic intuition behind Theorem 6 is as follows. Finite samples force investors to downweight directions of factor risk that are difficult to estimate. This shrinkage intensifies as dimensionality increases. What matters for performance is therefore not the number of factors, but whether economically relevant risk premia concentrate in directions that survive shrinkage. Random Features spread risk premia across many weak directions and are poorly aligned. Learned features, by contrast, can rotate the factor space so that economically important variation loads on strong principal components. Alignment determines how much of the population Sharpe ratio survives finite-sample shrinkage. This naturally raises the question of how to estimate $\mu'\Sigma^{-1}\mu$ when P/T is non-negligible.

In low dimensions, $\mu'\Sigma^{-1}\mu$ is usually estimated by the (Gibbons et al., 1989) (henceforth, GRS) statistic. In high dimensions, however, this estimator is biased. Chernov et al. (2025) show that the bias can be corrected using the same effective

penalty $\hat{Z}_*(z)$ that governs limits to learning:

$$\hat{W}^{debiased}(z) = \frac{\bar{F}'(zI + \hat{\Sigma})^{-1}\bar{F} - (z^{-1}\hat{Z}_*(z) - 1)}{z^{-1}\hat{Z}_*(z)} \approx \mu'(\hat{Z}_*(z)I + \Sigma)^{-1}\mu. \quad (56)$$

The debiased statistic $\hat{W}^{debiased}(z)$ therefore provides a natural finite-sample measure of economically relevant alignment. In the empirical analysis below, we apply this statistic to both random features and PTK-based learned features.

6. Empirics

This section evaluates the empirical implications of the Portfolio Tangent Kernel (PTK) framework using U.S. equity data. We compare the performance of three classes of stochastic discount factors (SDFs): (i) fully trained deep neural network (DNN) SDFs, (ii) random-feature SDFs following [DKKM](#), and (iii) PTK-based SDFs that optimally price features learned by the DNN. Our analysis focuses on out-of-sample pricing performance, factor spanning, statistical robustness, and economic alignment with long-horizon consumption risk. All results are reported using rolling-window implementations and are evaluated separately across market capitalization groups.

6.1. Data and Random Feature Portfolios

We use monthly data on 153 stock characteristics for U.S. equities over the period 1963–2024 from ([Jensen, Kelly, and Pedersen, 2023](#), hereafter JKP). This open-source dataset collects a comprehensive set of firm-level characteristics drawn from the asset

pricing literature.⁵ The universe consists of NYSE, AMEX, and NASDAQ stocks with CRSP share codes 10–12.

To construct a stock-month panel with a stable set of characteristics, we follow the filtering procedure of [Didisheim et al. \(2024\)](#). First, we restrict attention to a subset of 132 characteristics with fewer than 30% missing observations over the full sample. Second, we exclude nano stocks whose market capitalization falls below the 1st percentile of NYSE market capitalizations. Third, we drop stock-month observations with missing values for more than one-third of the characteristics. Finally, each characteristic is rank-standardized cross-sectionally to the interval $[-0.5, 0.5]$, and remaining missing values are imputed using the cross-sectional median, which equals zero by construction. After this procedure, the resulting $N_t \times 132$ matrix of characteristics at each date constitutes the conditioning set X_t . Let $R_{t+1} = (R_{i,t+1})_{i=1}^{N_t}$ denote the vector of next-month excess returns.

To isolate the role of statistical complexity while abstracting from liquidity concerns, we conduct the analysis separately across market capitalization groups. Following JKP, we study mega (largest 20%), large (80%–50%), small (50%–20%), and micro (20%–1%) stocks based on NYSE breakpoints each period, as well as the full universe of stocks above the 1% NYSE cutoff, which we refer to as “all.”

As a benchmark, we construct random-feature-based SDFs following [Didisheim et al. \(2024\)](#). Specifically, we define

$$S_t = \text{ReLU}(X_t \gamma) \in \mathbb{R}^{N_t \times P},$$

where $\gamma = (\gamma_{j,k}) \in \mathbb{R}^{d \times P}$ with $P = 25,000$ and $\gamma_{j,k} \sim N(0, 1)$ independently. These

⁵JKP characteristics for individual stocks are available at <https://wrds-www.wharton.upenn.edu/pages/get-data/contributed-data-forms/global-factor-data/>. Detailed documentation and additional factor portfolio data are provided at jkpfactors.com.

nonlinear transformations constitute the *random features* used in the analysis of [Didisheim et al. \(2024\)](#). We then form factor returns

$$F_{t+1}^{\text{ReLU}} = N_t^{-1/2} R'_{t+1} S_t. \quad (57)$$

Normalization. Throughout the paper, all factor returns are normalized by $N_t^{-1/2}$, where N_t denotes the cross-sectional sample size at time t . This scaling ensures that factor second moments remain stable across time and are comparable across periods with different cross-sectional dimensions. For the DNN and PTK constructions, this normalization is implemented directly in the factor definitions, ensuring scale comparability with the random feature benchmark.

We use these normalized factor returns to estimate a candidate stochastic discount factor as the return on a Markowitz portfolio computed over a rolling window of T months with ridge penalty z :

$$R_{t+1}^{\text{ReLU}}(z, T) = \theta_t(z; T)' F_{t+1}^{\text{ReLU}}, \quad (58)$$

where $\theta_t(z; T)$ is obtained from [\(22\)](#).

Because the impact of the ridge penalty depends on its magnitude relative to the factor spectrum, we scale z proportionally to the average eigenvalue of the factor covariance matrix,

$$z = z_{eff} P^{-1} \text{tr}(F' F / T). \quad (59)$$

We consider rolling estimation windows $T = 12, 60, 120, 240, 360$ months and set $z_{eff} = 10^{-5}$. The resulting ReLU-based SDF returns are combined into an ensemble benchmark using weights inversely proportional to their realized volatility over the

preceding 12 months. Importantly, these volatility weights are computed using only information available at time t .

Formally, the ensemble random feature SDF is given by

$$\bar{R}_{t+1}^{\text{ReLU}}(z_{eff}) = \sum_{T \in \{12, 60, 120, 240, 360\}} \omega_{t,T} R_{t+1}^{\text{ReLU}}(z_{eff} P^{-1} \text{tr}(F'F/T), T), \quad (60)$$

where $\omega_{t,T} \propto \hat{\sigma}_{t,T}^{-1}$ and $\hat{\sigma}_{t,T}$ denotes the realized volatility of $R_{\tau+1}^{\text{ReLU}}(z_{eff}, T)$, $\tau \leq t-1$, computed over the trailing 12 months.

6.2. Training the DNN and the PTK-SDF

We consider a multilayer perceptron (MLP) architecture with an input layer of dimension 132, corresponding to the number of characteristics, followed by D hidden layers of width w . We consider depths $D = 1, 2, 4$ and widths $w = 16, 32, 64, 128, 256, 512$. The parameter w controls network width, while D controls depth. We denote the resulting function family by $f(X; \theta; D; w)$.

The dimension of the parameter vector θ is given by

$$P = \underbrace{(d_{in} + 1) \times w}_{\text{Input layer}} + \underbrace{(D - 1) \times (w + 1) \times w}_{\text{Hidden layers}} + \underbrace{(w + 1)}_{\text{Output layer}},$$

see Definition 3. The deepest and widest network in our analysis contains 856,577 parameters. Our DNN–PTK duality implies that P is the relevant measure of model complexity: it equals both the number of factors in the associated PTK and the interpolation capacity of the network. In particular, by Theorem 3, the DNN-SDF can interpolate T observations—achieving in-sample returns equal to one for $t = 1, \dots, T$ —if and only if $P \geq T$, provided the PTK matrix is non-degenerate.

We train the DNN using gradient descent as described in Appendix B, employing

a rolling estimation window of 60 months and a learning rate $\eta = 2^{-16}$. At each time t , the network is trained using only data available up to time t . To obtain a common set of learned features, we train the network on the full stock universe (“all”) and obtain a time- t parameter vector θ_t^* . We then evaluate the resulting SDF separately on each size group (mega, large, small, and micro).

The resulting DNN-based SDF is

$$R_{t+1}^{\text{DNN}}(D; w) = N_t^{-1/2} R'_{t+1} f(X_t; \theta_t^*; D; w). \quad (61)$$

We next construct SDFs based on the Portfolio Tangent Kernel. Given a trained network $f(X; \theta_t^*; D; w)$, we form factor returns using the learned gradient features as in (39),

$$F_{\tau+1}(\theta_t^*; D; w) = N_\tau^{-1/2} R'_{\tau+1} \underbrace{\nabla_{\theta} f(X_\tau; \theta_t^*; D; w)}_{\text{time-}t \text{ learned features}} \in \mathbb{R}^P, \quad P = \dim(\theta). \quad (62)$$

Thus, $F_{\tau+1}(\theta_t^*; D; w)$ represents returns on the time- t vintage of characteristics-managed portfolios constructed from features learned at time t .

The corresponding time- t Portfolio Tangent Kernel (PTK) is defined as

$$\mathbb{K}((R_{\tau_1+1}, X_{\tau_1}); (R_{\tau_2+1}, X_{\tau_2}); \theta_t^*; D; w) = F_{\tau_1+1}(\theta_t^*; D; w)' F_{\tau_2+1}(\theta_t^*; D; w). \quad (63)$$

Fixing a rolling window length T , we estimate the ridge-penalized Markowitz portfolio (22) using factor returns $F_{\tau+1}(\theta_t^*; D; w)$ for $\tau = t - T, \dots, t - 1$. Denoting the resulting portfolio weights by $\Theta_t(\theta_t^*; D; w; T; z)$, the associated PTK-based SDF is

$$R_{t+1}^{\text{PTK}}(D; w; T; z) = \Theta_t(\theta_t^*; D; w; T; z)' F_{t+1}(\theta_t^*; D; w). \quad (64)$$

As with the random feature benchmark, we construct an ensemble PTK-SDF by averaging across rolling windows,

$$\bar{R}_{t+1}^{\text{PTK}}(D; w; z_{eff}) = \sum_{T \in \{12, 60, 120, 240, 360\}} \omega_{t,T}^{\text{PTK}} R_{t+1}^{\text{PTK}}(D; w; T; z_{eff} P^{-1} \text{tr}(F'F/T)), \quad (65)$$

where $\omega_{t,T}^{\text{PTK}} \propto \hat{\sigma}_{t,T}^{-1}$ and $\hat{\sigma}_{t,T}$ denotes the realized volatility of $R_{\tau+1}^{\text{PTK}}(D; w; T; z)$, $\tau \leq t - 1$, computed over the trailing 12 months.

This aggregation exploits potential non-stationarity by allowing the pricing rule to adapt across time scales. Importantly, this flexibility is specific to the PTK construction and entails negligible computational cost, as estimating Markowitz portfolios for multiple T is numerically trivial.

6.3. The Performance of R^{ReLU} , R^{DNN} , and R^{PTK}

The random feature portfolios R_{t+1}^{ReLU} defined in (58) correspond to a shallow (single hidden layer) neural network with fixed, randomly generated features. Despite the absence of feature learning, [Didisheim et al. \(2024\)](#) document strong empirical performance of this nonlinear SDF, driven by sheer statistical complexity: the model effectively prices returns using $P = 25,000$ nonlinear factors. This benchmark, therefore, provides a stringent baseline against which to evaluate fully trained neural networks.

Our first objective is to assess whether a trained DNN is able to *learn economically relevant features*—that is, priced nonlinear characteristics that are not already captured by the random feature SDF.

Figure 2 reports the Sharpe ratios of $R_{t+1}^{\text{DNN}}(D; w)$ from (61) as a function of network depth and width, alongside the ensembled ReLU benchmark $\bar{R}^{\text{ReLU}}$ from

(60). The patterns are similar across size groups; we therefore focus on the full stock universe (“all”).

Several findings stand out. First, depth is essential. Shallow networks ($D = 1$) perform poorly and substantially underperform the random feature benchmark. Second, deeper networks are capable of extracting more structure: For example, the depth-2, width-256 network attains a Sharpe ratio of 3.8, slightly exceeding the ReLU benchmark (3.7). This architecture is highly complex, with $P = 100,097$ trained parameters. Nevertheless, even such large networks generally fail to outperform the random feature SDF across all size groups, despite having four times as many parameters.

Taken at face value, these results might suggest that fully trained DNNs add little value for asset pricing. This conclusion, however, is misleading. As emphasized by [Kelly and Malamud \(2025\)](#), raw Sharpe ratios alone are not an appropriate metric for evaluating machine learning models. What matters is whether the model uncovers *new priced variation*—that is, whether it discovers risk premia that are unspanned by existing models.

To separate *what* the DNN learns from *how* it prices, we test the spanning hypothesis by estimating the univariate regressions on the (60) benchmark with $z_{eff} = 10^{-5}$:

$$R_{t+1}^{\text{DNN}}(D; w) = \alpha + \beta \bar{R}_{t+1}^{\text{ReLU}}(10^{-5}) + \varepsilon_{t+1}. \quad (66)$$

Figure 3 reports the t -statistics of the intercepts.

The results strongly support the feature-learning hypothesis. For moderately complex deep architectures (e.g., $D = 2$ and $D = 4$ with $w = 256$), the estimated alpha is positive and highly statistically significant. In contrast, very small networks

fail to uncover new priced characteristics, while extremely large networks (depth 4 with a width above 256, corresponding to $P > 200,000$) sometimes fail to do so reliably.⁶

Economically, these findings imply that moderately complex DNNs ($P \approx 100,000$) are effective at learning *what to price*: they discover a rich set of priced nonlinear characteristics that are not spanned by random features. However, they are less effective at learning *how to price* these characteristics, resulting in suboptimal Sharpe ratios despite strong alpha.

Motivated by these results, we focus on the architecture $R^{\text{DNN}}(D = 2, w = 256)$ in the remainder of the analysis. This specification exhibits the strongest and most robust feature-learning performance across all size groups, as illustrated in Figure 3.

We now turn to our central theoretical prediction: the Portfolio Tangent Kernel (PTK) should optimally price the features learned by the DNN. If the learned characteristics subsume those captured by the ReLU benchmark, and if the PTK implements an efficient pricing rule, then $\bar{R}^{\text{PTK}}(D = 2, w = 256)$ should outperform both R^{DNN} and $\bar{R}^{\text{ReLU}}$.

Figure 4 confirms this prediction. The PTK-SDF delivers substantially higher Sharpe ratios than both benchmarks across all size groups. The gains are particularly pronounced for mega and large stocks: PTK achieves Sharpe ratios of 1.6 and 1.9, compared to 1.3 and 1.6 for the ReLU benchmark.

A striking implication is that the *ridgeless* PTK attains the highest Sharpe ratios across all size groups. These findings underscore the role of statistical complexity: when low-variance principal components contain an economically meaningful

⁶Training such large networks may require alternative optimization schemes or architecture-adjusted hyperparameters; see Yang et al. (2022). We leave this issue for future work.

signal—as suggested by the strong alpha evidence in Figure 3—explicit shrinkage becomes unnecessary.

Figure 5 further shows that the outperformance of PTK is statistically significant. The PTK-SDF delivers large and significant alpha relative to both R^{DNN} and $\bar{R}^{\text{ReLU}}$, indicating that it fully subsumes their pricing information.

We therefore conclude that feature learning and pricing should be viewed as distinct tasks. DNNs excel at discovering rich nonlinear characteristics, while the PTK provides a statistically efficient mechanism for pricing them. The PTK representation renders the learned SDF amenable to the Random Matrix Theory analysis developed in Section 5, to which we now turn.

This out-performance is also statistically significant, as is shown by Figure 5. We therefore conclude that learned features (PTK) entirely subsume the performance of the DNN. In turn, the PTK representation of the candidate SDF makes it amenable to the RMT analysis from Section 5.

6.4. The Dynamics of Spectral Complexity

By Theorem 5, a central determinant of LFM performance is the effective shrinkage $\hat{Z}_*(z)$ defined in (54). In the PTK setting, we adopt a scaled version of (59) and set

$$z = z_{\text{eff}} T^{-1} \text{tr}(FF'/T), \quad (67)$$

which is proportional to (59). Since $\hat{Z}_*(z)$ scales linearly with the kernel matrix $K = FF'$, we normalize the effective ridge penalty and define

$$\frac{\hat{Z}_*(z)}{T^{-2} \text{tr}(K)} = \frac{1}{T^{-1} \text{tr}((z_{\text{eff}} T^{-2} \text{tr}(K) I + K/T)^{-1}) \cdot T^{-2} \text{tr}(K)}. \quad (68)$$

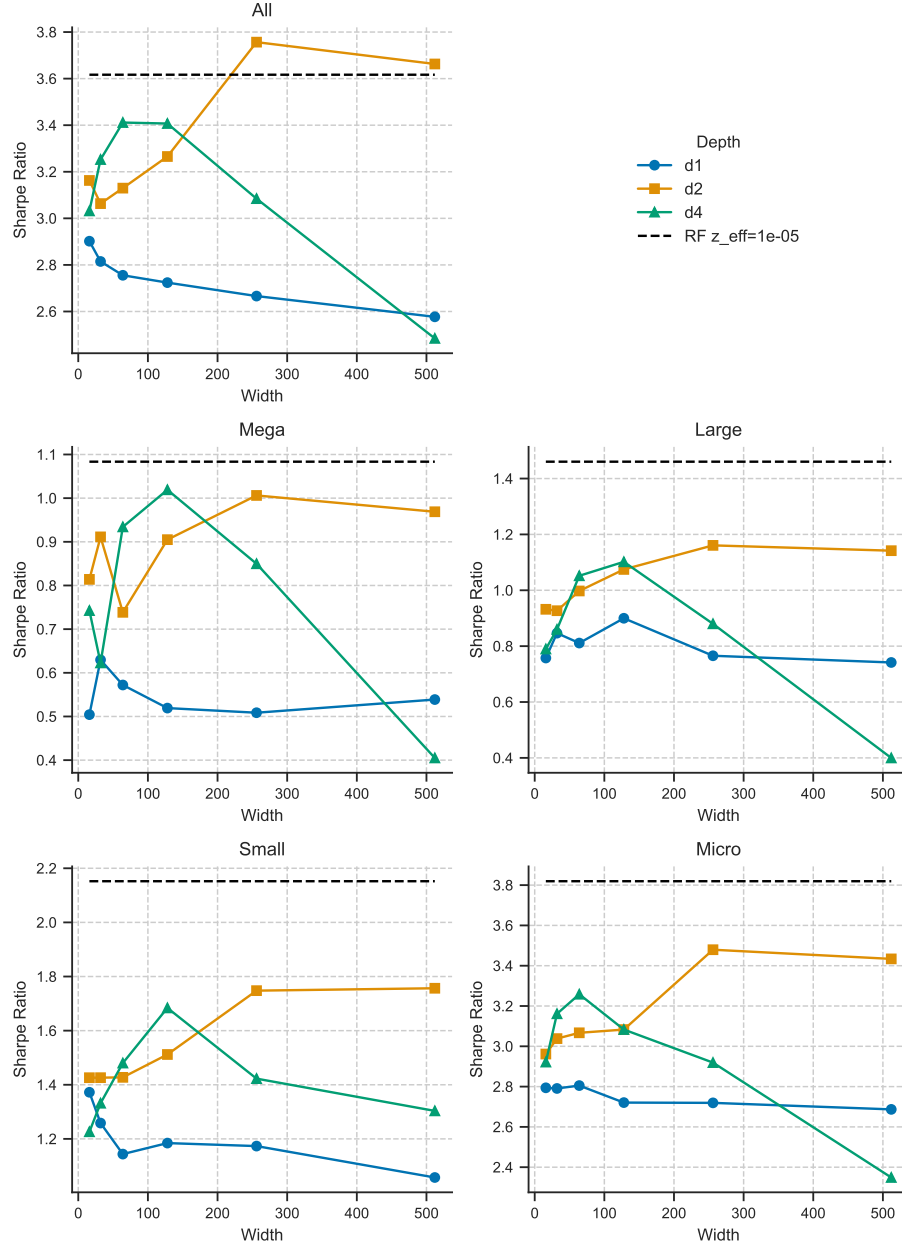


Figure 2: Sharpe Ratio of R^{DNN} from (61) as a Function of Model Complexity (depth D and width w). The lines $d1, d2, d4$ correspond to $D = 1, 2$, and 4 , respectively, with widths $w \in \{2^4, \dots, 2^9\}$. The dashed line is the Sharpe ratio of the $\bar{R}^{\text{ReLU}}$ benchmark from (60) with $z_{\text{eff}} = 10^{-5}$.

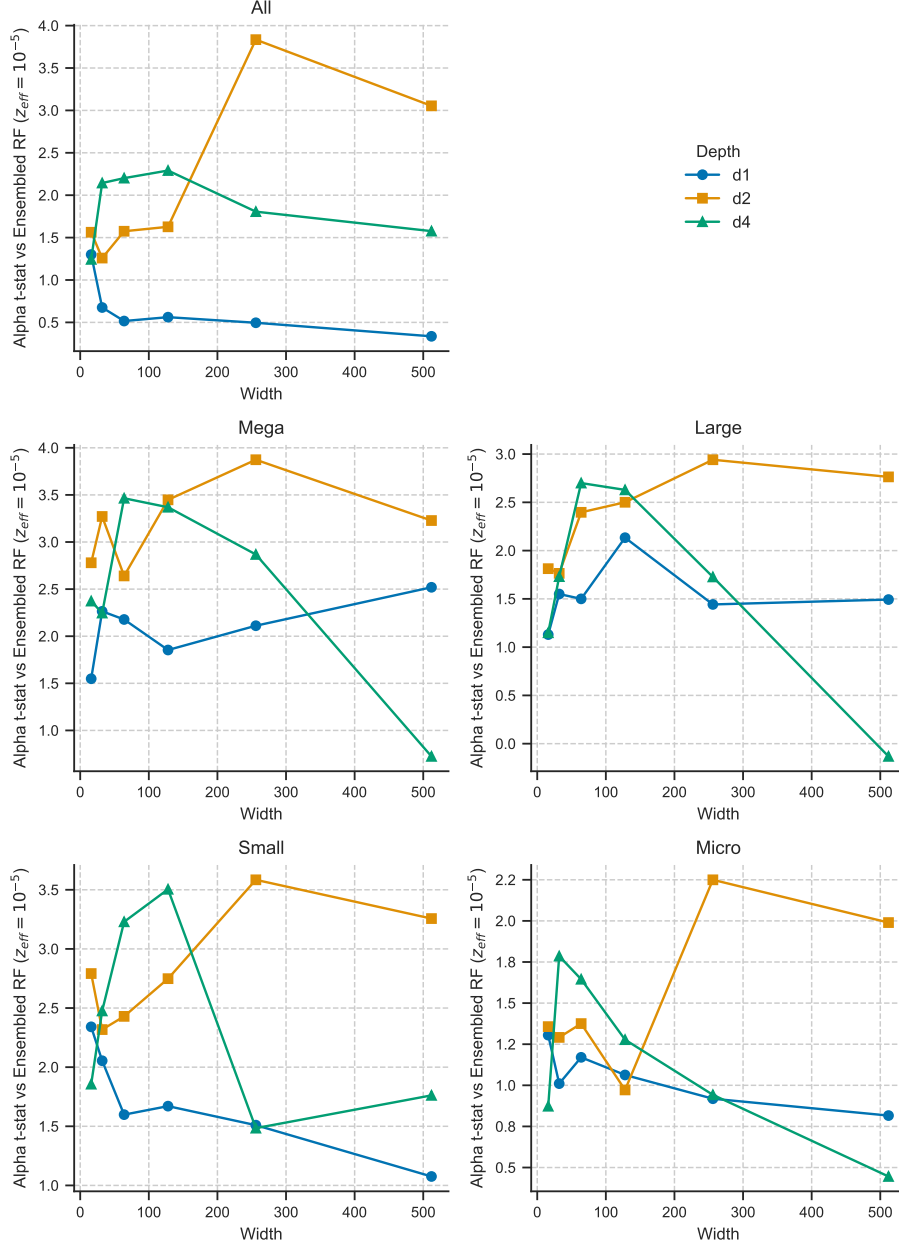


Figure 3: Alpha t-statistics from the regressions (61) as a Function of Model Complexity (depth D and width w). The lines $d1, d2, d4$ correspond to $D = 1, 2$, and 4 , respectively, with widths $w \in \{2^4, \dots, 2^9\}$.

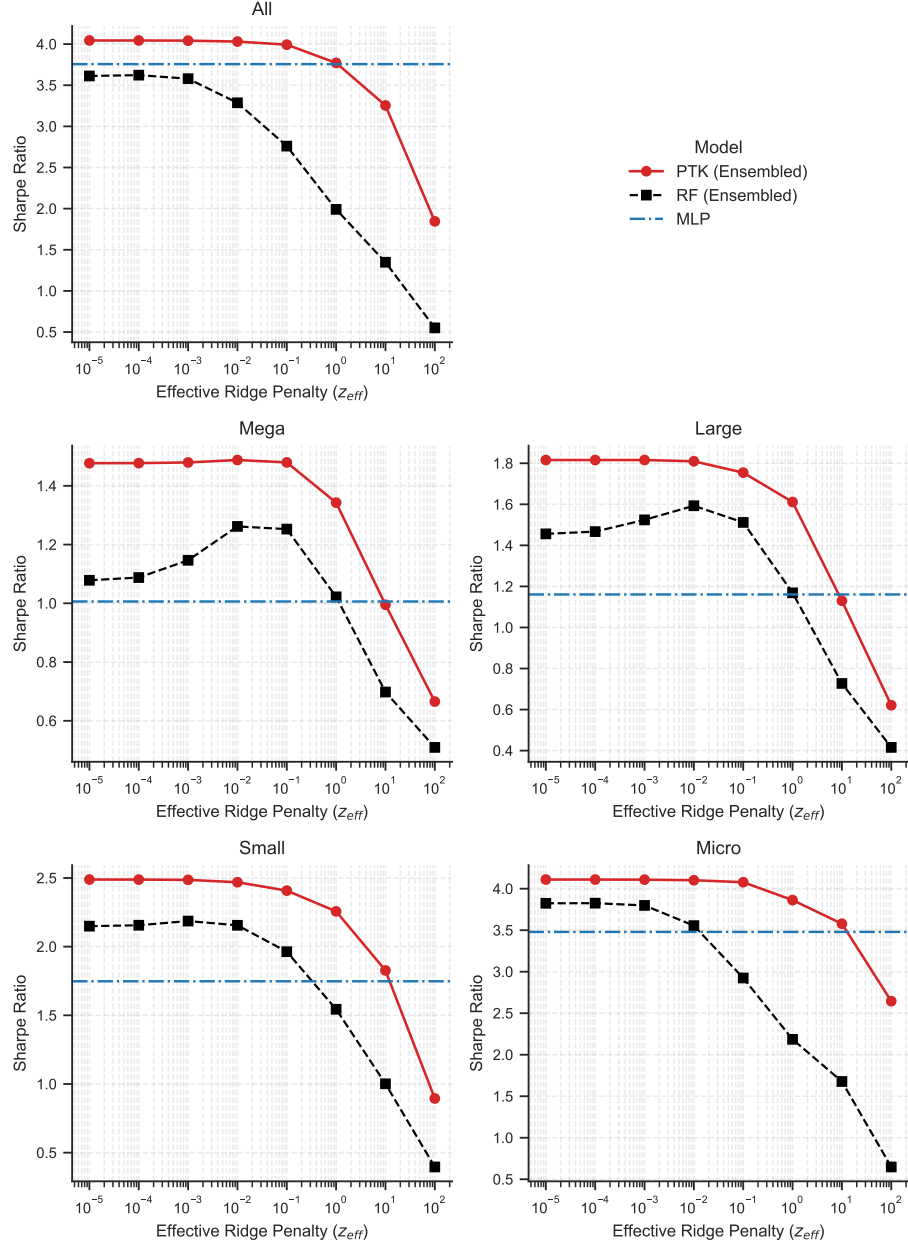


Figure 4: Sharpe Ratios of $\bar{R}^{\text{PTK}}(D=2, w=256; z_{eff})$ from (65) and $\bar{R}^{\text{ReLU}}(z_{eff})$ from (60) (both are ensembled across T), as a function of the z_{eff} . The blue dash-dotted line represents the Sharpe Ratio of $R^{\text{DNN}}(D=2, w=256)$ benchmark from (61).

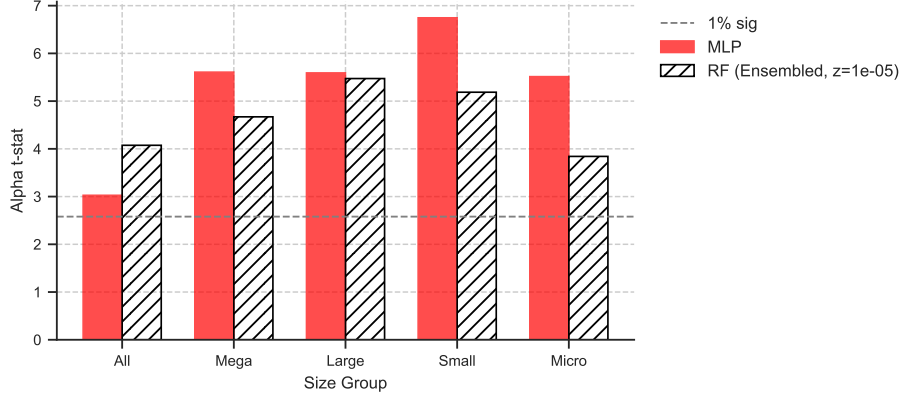


Figure 5: The t-statistics of alpha from univariate regressions of $\bar{R}^{\text{PTK}}(D = 2, w = 256)$ with $z_{\text{eff}} = 10^{-5}$ (see (65)) on $R^{\text{DNN}}(D = 2, w = 256)$ from (61) (red bars) and $\bar{R}^{\text{ReLU}}$ from (60) (grey striped bars). The dashed horizontal line indicates the level of 1% statistical significance.

By Jensen’s inequality, $T^{-1} \text{tr}(A^{-1}) \geq (T^{-1} \text{tr}(A))^{-1}$ for any symmetric positive definite $T \times T$ matrix A , which implies

$$\frac{\hat{Z}_*(z_{\text{eff}} T^{-2} \text{tr}(K))}{T^{-2} \text{tr}(K)} \leq 1 + z_{\text{eff}}. \quad (69)$$

The normalized quantity in (68) measures the fraction of total factor variance effectively shaded by implicit regularization in an over-parameterized LFM. The upper bound in (69) is attained when $K \approx I$, corresponding to maximal spectral complexity, where all principal components contribute equally. We therefore refer to (68) as the *spectral complexity* of the kernel matrix.

Figure 6 reports the time-series dynamics of spectral complexity (left panel) and the debiased GRS statistic (56) (right panel) for both random-feature-based factors (57) and PTK-based factors. Several striking empirical regularities emerge.

First, PTK factors exhibit spectral complexity roughly an order of magnitude larger than that of random features. Rather than concentrating risk in a small num-

ber of dominant directions, the learned representation spreads economically relevant variation across a large number of principal components. This finding underscores that feature learning does not simplify the factor space; instead, it reorganizes it.

Second, PTK spectral complexity rises monotonically after 2000 and increases by approximately a factor of six over the sample period. In contrast, the spectral complexity of random feature factors remains essentially flat. The increase is therefore specific to learned, priced features and reflects a gradual rise in the dimensionality of economically relevant risk.

Third, the debiased GRS statistic (56) rises steadily until the early 2000s and declines after 2010. Importantly, this decline is not accompanied by a deterioration in realized SDF performance (Figure 9 in the Appendix). Combined with the sharp increase in $\hat{Z}_*(z)$, this pattern suggests that the population Sharpe ratio $\mu'\Sigma^{-1}\mu$ has continued to grow, while finite-sample limits to learning have intensified due to rising spectral complexity.

To further investigate alignment in the sense of Theorem 6, we conduct a principal-component truncation exercise. For each K , we construct truncated versions of $R_{t+1}^{\text{PTK}}(D_*; w_*; T, z)$ and $R_{t+1}^{\text{ReLU}}(T, z)$ by retaining only the top K eigenvectors of the sample covariance matrix

$$\hat{\Sigma}_t = T^{-1} \sum_{\tau=t-T}^{t-1} F_{\tau+1} F_{\tau+1}' = \hat{U}_t \hat{D}_t \hat{U}_t', \quad (70)$$

and forming portfolios based on

$$\hat{F}_t(K) = \hat{U}_t[:, :K]' \hat{F}_t. \quad (71)$$

Figure 7 plots Sharpe ratios as a function of K for learned gradient features and

random features. Consistent with the alignment mechanism of Section 5, learned features concentrate economically relevant signal in far fewer directions. For the full stock universe, the top 15 (out of 60) PTK principal components already achieve a Sharpe ratio of 3.8, close to the full-feature maximum of 3.9. By contrast, random features require substantially larger K to extract the signal and reach only 2.3 at the same truncation level.

Taken together, these results highlight a central tension. Feature learning increases spectral complexity and exacerbates limits to learning, yet simultaneously improves alignment by concentrating the economic signal in statistically robust directions. The empirical success of PTK-based pricing stems precisely from this trade-off.

6.5. PTK Alignment with Consumption Risk

Modern asset pricing theories emphasize long-run and future consumption risk as key drivers of equity premia (Bansal and Yaron, 2004; Campbell et al., 2018). In these models, the stochastic discount factor (SDF) loads on future consumption realizations rather than contemporaneous consumption growth.

Parker and Julliard (2005) show that covariances with future realized consumption are priced in the cross-section of stock returns, while Malamud et al. (2025) provide a micro-founded version based on financial frictions. These frameworks offer economically disciplined benchmarks for evaluating whether machine-learning-based SDFs capture meaningful macroeconomic risk.

Our alignment framework implies that economically relevant pricing information should concentrate in the leading principal components of the factor space (Theorem 6). We therefore examine whether the dominant components of PTK-based factors exhibit stronger alignment with future consumption risk than those of random features or fully trained DNN-SDFs.

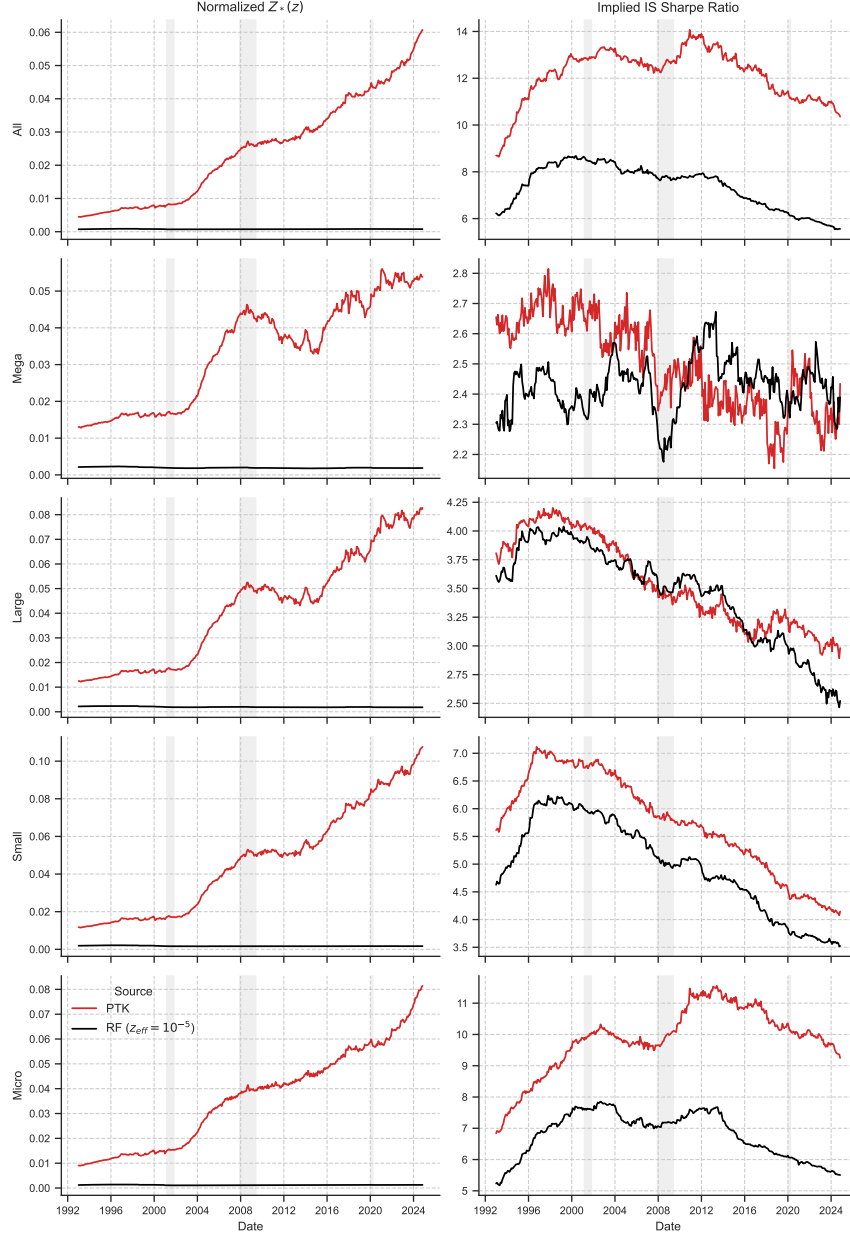


Figure 6: The dynamics of spectral complexity (68) and the debiased GRS statistic (56) estimated using $T = 360$ for R^{PTK} and R^{ReLU} ; we use $D = 2$, $w = 256$.

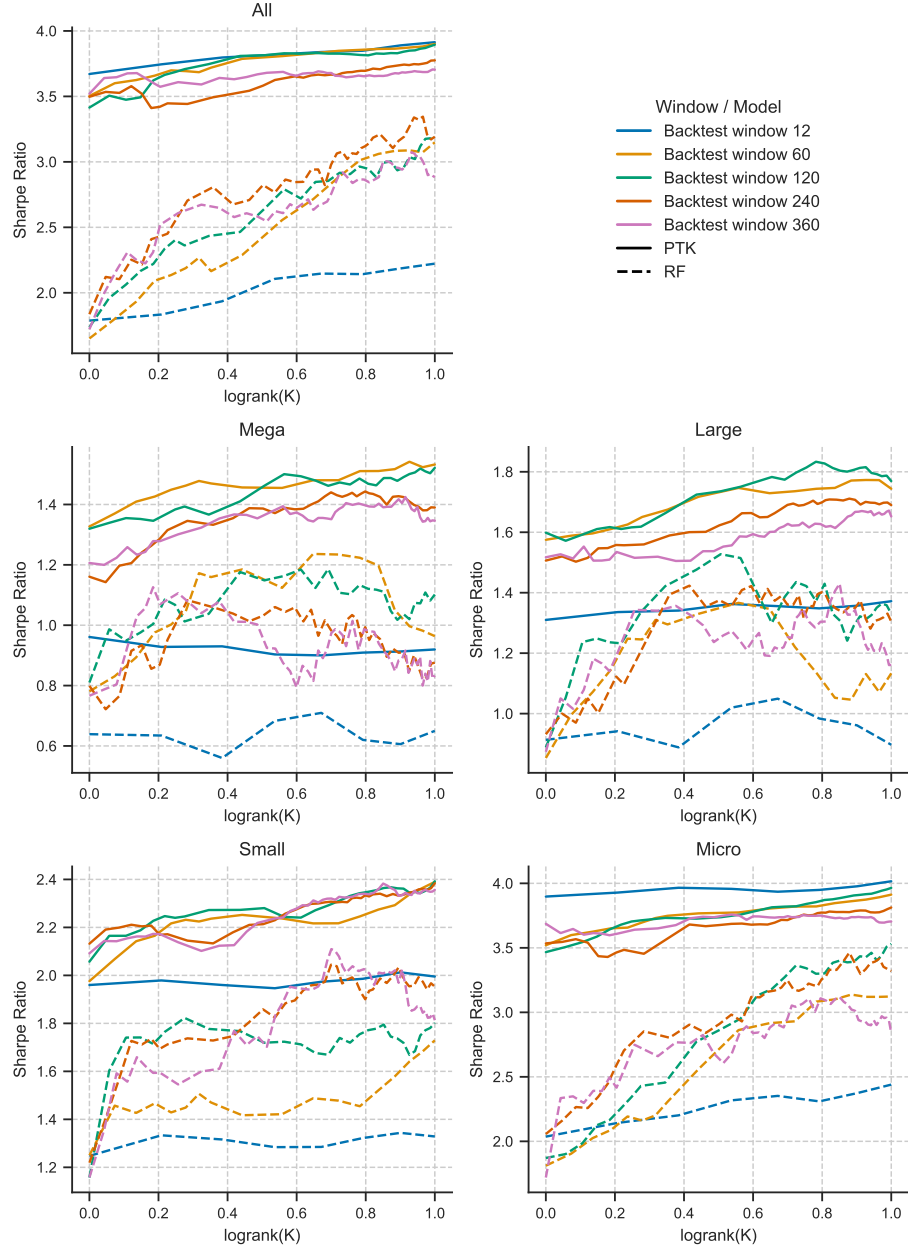


Figure 7: Factor Alignment. This figure plots the Sharpe Ratio against the number of principal components (K) for PTK Factors and random feature factors, penalized with $z_{eff} = 10^{-5}$, for different backtest window sizes $T = 12, 60, 120, 240, 360$.

We project $\bar{R}^{\text{PTK}}$ and $\bar{R}^{\text{ReLU}}$ onto their top two principal components (71) and compute correlations and predictive regressions with future consumption growth, following Parker and Julliard (2005) and Malamud et al. (2025).⁷

Figure 8 reports the results. Across all size groups, the PTK-SDF exhibits substantially stronger alignment with both consumption-based benchmarks than either the DNN-SDF or the random-feature benchmark. PTK delivers larger absolute correlations, higher regression t -statistics, and materially greater explanatory power. For the aggregate stock universe, the projected PTK-SDF is approximately -30% correlated with future consumption growth under Parker and Julliard (2005) and nearly -50% under Malamud et al. (2025), compared to correlations around -10% and -20% , respectively, for the alternatives.

These findings highlight the economic role of PTK. Gradient-based feature learning rotates the high-dimensional characteristic space so that consumption-related risk loads on statistically strong directions that survive shrinkage. Random features fail to achieve this alignment, while unconstrained DNN-SDFs do not price it efficiently.

Overall, PTK isolates the economically relevant component of feature learning and translates it into a disciplined pricing rule, yielding tradable SDFs that are tightly linked to long-horizon consumption risk.

7. Conclusion

This paper studies how modern machine learning models price assets in high-dimensional environments and why their performance is constrained by fundamental limits to learning. We develop a framework that links deep neural networks, kernel methods,

⁷Results without principal-component projection are weaker, indicating that lower-variance PTK components primarily capture variation orthogonal to consumption risk. See Figure 10 in the Appendix.

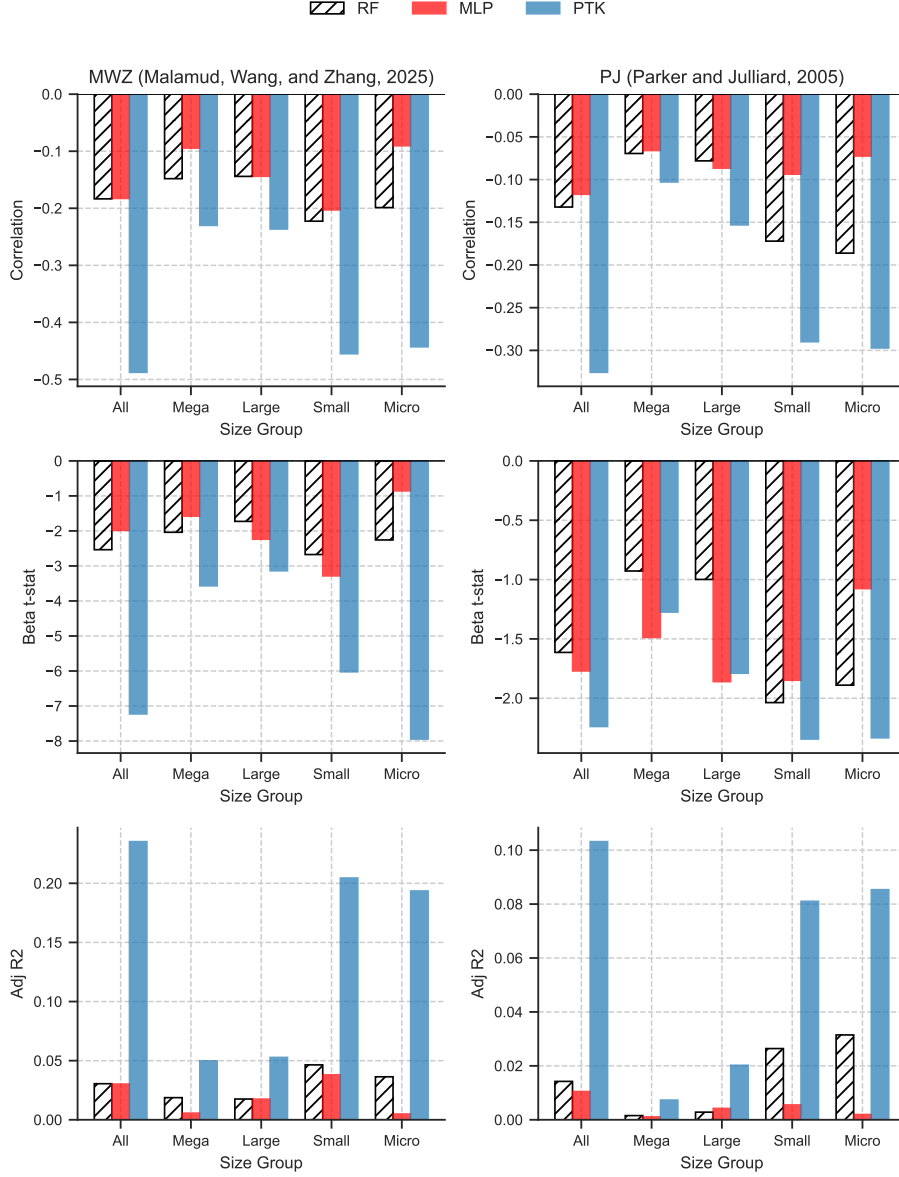


Figure 8: (Top 2 PCs for ReLU and PTK based SDFs) Machine Learning SDFs and Consumption Risk. This figure reports the relation between tradeable machine-learning stochastic discount factors (SDFs) and consumption-based SDFs from [Parker and Julliard \(2005\)](#) (PJ) and [Malamud et al. \(2025\)](#) (MWZ). The top row shows time-series correlations between portfolio returns from the fully trained DNN SDF R^{DNN} (61) (red), the random-feature benchmark $\bar{R}^{\text{ReLU}}$ (60) (grey, striped), and the PTK-based SDF $\bar{R}^{\text{PTK}}$ (65) (blue), and the MWZ future-consumption SDF (left panel) and the PJ long-run consumption SDF (right panel). The middle row reports t -statistics from univariate time-series regressions of each consumption-based SDF on the corresponding tradeable SDF return. The bottom row reports the associated adjusted R^2 . All statistics are shown separately by size group. Consumption-based SDFs are constructed using future realized consumption growth aggregated over horizons of 84 months (MWZ) and 36 months (PJ), respectively, which truncates the sample in June 2018. Ridge regularization with $z_{\text{eff}} = 10^{-5}$ is applied to the ReLU and PTK portfolios.

and factor pricing through the Portfolio Tangent Kernel (PTK), which provides an explicit representation of the pricing rule induced by gradient-based training.

Our main conceptual contribution is to separate feature learning from pricing. Fully trained neural networks are effective at discovering rich, nonlinear characteristics that are unspanned by standard factor constructions. However, when used directly as stochastic discount factors, these models often fail to aggregate the learned features efficiently. The PTK makes this distinction explicit: network architecture and training determine which features are learned, while the PTK governs how these features are priced through a statistically disciplined factor model.

Using random matrix theory, we show that finite-sample estimation induces endogenous shrinkage even in ridgeless, interpolating regimes. This shrinkage imposes an effective low-dimensional structure on the pricing rule, and its severity depends critically on the alignment between risk premia and the principal components of the feature space. We formalize this mechanism through a projection theorem that clarifies how limits to learning translate into economic restrictions on achievable Sharpe ratios.

Empirically, we find that learned gradient features exhibit substantially stronger alignment than random features, allowing the PTK-based SDF to outperform both raw DNN-SDFs and strong random feature benchmarks across asset size groups. The results show that the gains from deep learning arise not from abandoning structure, but from combining flexible representation learning with explicit and statistically efficient pricing rules.

Overall, our findings suggest that the role of machine learning in asset pricing is best understood as expanding the space of economically relevant characteristics, while leaving pricing to be governed by well-understood principles of risk, regularization, and finite-sample discipline.

References

- BANSAL, R. AND A. YARON (2004): “Risks for the long run: A potential resolution of asset pricing puzzles,” The journal of Finance, 59, 1481–1509.
- BARTLETT, P. (2003): “Reproducing kernel Hilbert spaces,” Lecture Notes, CS281B/Stat241B, Statistical Learning Theory.
- BELKIN, M. (2021): “Fit without fear: remarkable mathematical phenomena of deep learning through the prism of interpolation,” Acta Numerica, 30, 203–248.
- CAMPBELL, J. Y., S. GIGLIO, C. POLK, AND R. TURLEY (2018): “An intertemporal CAPM with stochastic volatility,” Journal of Financial Economics, 128, 207–233.
- CHEN, L., M. PELGER, AND J. ZHU (2024): “Deep learning in asset pricing,” Management Science, 70, 714–750.
- CHEN, Z., B. KELLY, AND S. MALAMUD (2025): “Limits To (Machine) Learning,” arXiv preprint arXiv:2512.12735.
- CHERNOV, M., B. T. KELLY, S. MALAMUD, AND J. SCHWAB (2025): “A Test of the Efficiency of a Given Portfolio in High Dimensions,” Tech. rep., National Bureau of Economic Research.
- COCHRANE, J. H. (2011): “Presidential address: Discount rates,” The Journal of finance, 66, 1047–1108.
- CONG, L. W., G. FENG, J. HE, AND X. HE (2025): “Growing the efficient frontier on panel trees,” Journal of Financial Economics, 167, 104024.
- DIDISHEIM, A., S. B. KE, B. T. KELLY, AND S. MALAMUD (2024): “APT or “AIPT”? The Surprising Dominance of Large Factor Models,” Tech. rep., National Bureau of Economic Research.

- FAN, J., Z. T. KE, Y. LIAO, AND A. NEUHERL (2022): “Structural Deep Learning in Conditional Asset Pricing,” Available at SSRN 4117882.
- GIBBONS, M. R., S. A. ROSS, AND J. SHANKEN (1989): “A test of the efficiency of a given portfolio,” Econometrica: Journal of the Econometric Society, 1121–1152.
- GU, S., B. KELLY, AND D. XIU (2020a): “Autoencoder Asset Pricing Models,” Journal of Econometrics.
- (2020b): “Empirical asset pricing via machine learning,” The Review of Financial Studies, 33, 2223–2273.
- HARVEY, C. R., Y. LIU, AND H. ZHU (2016): “... and the cross-section of expected returns,” The Review of Financial Studies, 29, 5–68.
- HOU, K., C. XUE, AND L. ZHANG (2020): “Replicating anomalies,” The Review of Financial Studies, 33, 2019–2133.
- JACOT, A., F. GABRIEL, AND C. HONGLER (2018): “Neural tangent kernel: Convergence and generalization in neural networks,” Advances in neural information processing systems, 31.
- JENSEN, T. I., B. KELLY, AND L. H. PEDERSEN (2023): “Is there a replication crisis in finance?” The Journal of Finance, 78, 2465–2518.
- KELLY, B., S. PRUITT, AND Y. SU (2020): “Characteristics are Covariances: A Unified Model of Risk and Return,” Journal of Financial Economics.
- KELLY, B. AND D. XIU (2023): “Financial Machine Learning,” Working Paper.
- KELLY, B. T., B. KUZNETSOV, S. MALAMUD, AND T. A. XU (2025): “Artificial intelligence asset pricing models,” Tech. rep., National Bureau of Economic Research.
- KELLY, B. T. AND S. MALAMUD (2025): “Understanding the virtue of complexity,” Available at SSRN 5346842.

- KELLY, B. T., S. MALAMUD, M. POURMOHAMMADI, AND F. TROJANI (2023): “Universal Portfolio Shrinkage,” Swiss Finance Institute Research Paper.
- KELLY, B. T., S. MALAMUD, AND K. ZHOU (2022): “The Virtue of Complexity Everywhere,” Available at SSRN.
- KOZAK, S., S. NAGEL, AND S. SANTOSH (2020): “Shrinking the cross-section,” Journal of Financial Economics, 135, 271–292.
- LONG, P. M. (2021): “Properties of the after kernel,” arXiv preprint arXiv:2105.10585.
- MALAMUD, S., N. WANG, AND Y. ZHANG (2025): “Consumer Credit and Asset Prices,” Available at SSRN 4276714.
- MCLEAN, R. D. AND J. PONTIFF (2016): “Does academic research destroy stock return predictability?” The Journal of Finance, 71, 5–32.
- NAKKIRAN, P., G. KAPLUN, Y. BANSAL, T. YANG, B. BARAK, AND I. SUTSKEVER (2021): “Deep double descent: Where bigger models and more data hurt,” Journal of Statistical Mechanics: Theory and Experiment, 2021, 124003.
- PARKER, J. A. AND C. JULLIARD (2005): “Consumption Risk and the Cross Section of Expected Returns,” Journal of Political Economy, 113, 185–222.
- RAHIMI, A. AND B. RECHT (2007): “Random Features for Large-Scale Kernel Machines.” in NIPS, Citeseer, vol. 3, 5.
- ROSS, S. A. (1976): “The Arbitrage Theory of Capital Asset Pricing,” Journal of Economic Theory, 13, 341–360.
- SCHWAB, J., B. KELLY, S. MALAMUD, AND T. A. XU (2025): “Training NTK to Generalize with KARE,” arXiv preprint arXiv:2505.11347.

- WEI, A., W. HU, AND J. STEINHARDT (2022): “More than a toy: Random matrix models predict how real-world neural representations generalize,” in International conference on machine learning, PMLR, 23549–23588.
- YANG, G. (2020): “Tensor programs ii: Neural tangent kernel for any architecture,” arXiv preprint arXiv:2006.14548.
- YANG, G., E. J. HU, I. BABUSCHKIN, S. SIDOR, X. LIU, D. FARHI, N. RYDER, J. PACHOCKI, W. CHEN, AND J. GAO (2022): “Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer,” arXiv preprint arXiv:2203.03466.
- ZHOU, Y., T. PANG, K. LIU, C. H. MARTIN, M. W. MAHONEY, AND Y. YANG (2023): “Temperature Balancing, Layer-wise Weight Analysis, and Neural Network Training,” arXiv preprint arXiv:2312.00359.

A. MLPs and the NTK in the Infinite Width Limit

Definition 3 (Multi-Layer Perceptron (MLP)) *Fix a neural network architecture given by the widths of layers $(n_1, \dots, n_L)$. Let $\theta = (W^{(0)}, b^{(0)}, \dots, W^{(L-1)}, b^{(L-1)})$ be a collection of weights and biases. Here, $W^{(l)} \in \mathbb{R}^{n_l \times n_{l+1}}$, $b^{(l+1)} \in \mathbb{R}^{n_l}$, and the total dimension of θ is $P = \sum_{l=0}^{L-1} (n_l + 1)n_{l+1}$. Let also $\phi : \mathbb{R} \rightarrow \mathbb{R}$ be a Lipschitz, twice differentiable nonlinearity with a bounded second derivative. The MLP neural*

network $f(x; \theta)$ is defined as

$$\begin{aligned}
x &= \text{input} \in \mathbb{R}^d \\
y^{(l)}(x) &= \begin{cases} x & \text{if } l = 0, \\ \phi(z^{(l)}(x)) & \text{if } l > 0. \end{cases} \\
z_i^{(l+1)}(x) &= \frac{1}{n_l^{1/2}} \sum_j W_{ij}^{(l)} y_j^{(l)}(x) + b_i^{(l)},
\end{aligned} \tag{72}$$

where $y^{(l)}(x), z^{(l)}(x) \in \mathbb{R}^{n_l}$. The output of the network is

$$f(x; \theta) = z^{(L)}(x) = \frac{1}{n_{L-1}^{1/2}} \sum_{j=1}^{n_{L-1}} W_j^{(L-1)} y_j^{(L-1)}(x). \tag{73}$$

At initialization, all weights $W_{i,j}^{(\ell)}$ and biases $b^{(l)}$ in each hidden layer are sampled independently from $N(0, (\sigma_W^{(l)})^2)$ and $N(0, (\sigma_b^{(l)})^2)$, respectively, for some layer-specific parameters $\sigma_W^{(l)}, \sigma_b^{(l)}$.

As a benchmark example, consider a single-hidden-layer NN

$$\begin{aligned}
f(x; \theta) &= z^1(x) = \frac{1}{n_1^{1/2}} \sum_{k=1}^{n_1} W_k^1 \phi \left(\frac{1}{n_0^{1/2}} \sum_{i=1}^d W_{k,i}^0 x_i + b_k^0 \right) \\
&= \frac{1}{n_1^{1/2}} \sum_{k=1}^{n_1} W_k^1 \phi \left(\frac{1}{n_0^{1/2}} (W_k^0)' x + b_k^0 \right) \\
&= \frac{1}{n_1^{1/2}} (W^1)' \phi \left(\frac{1}{n_0^{1/2}} (W^0)' x + b^0 \right).
\end{aligned} \tag{74}$$

The simple random feature model is therefore equivalent to the shallow (single hidden layer) neural network (74), with $\theta = W^1 \in \mathbb{R}^P$, $W_k = W_k^0 \in \mathbb{R}^d$, $P = n_1$. However, there is one important caveat: Neural networks are trained end-to-end; that is, every single weight, including all weights of all hidden layers, is optimized. E.g., for the

simple 1-hidden layer net (74), the whole $\theta = (W^1, W^0, b^0) \in \mathbb{R}^{n_1(d+2)}$ is optimized upon.

Definition 4 (The Infinite Width NTK) *Let*

$$\begin{aligned} & \widehat{\Sigma}^{(l)}(x, \tilde{x}) \\ &= (\sigma_w^{(l)})^2 \int dz d\tilde{z} \frac{d}{dz} \phi(z) \frac{d}{d\tilde{z}} \phi(\tilde{z}) \mathcal{N} \left(\begin{pmatrix} z \\ \tilde{z} \end{pmatrix}; \mathbf{0}, (\sigma_w^{(l-1)})^2 \begin{pmatrix} \Sigma^{(l-1)}(x, x) & \Sigma^{(l-1)}(x, \tilde{x}) \\ \Sigma^{(l-1)}(\tilde{x}, x) & \Sigma^{(l-1)}(\tilde{x}, \tilde{x}) \end{pmatrix} + (\sigma_b^{(l-1)})^2 \mathbf{1}_{2 \times 2} \right). \end{aligned} \quad (75)$$

Let also

$$\Theta^{(1)}(x, \tilde{x}) = \Sigma^{(1)}(x, \tilde{x}) + 1, \quad (76)$$

and then define recursively for $l > 1$:

$$\Theta^{(l)}(x, \tilde{x}) = \Theta^{(l-1)}(x, \tilde{x}) \widehat{\Sigma}^{(l)}(x, \tilde{x}) + \Sigma^{(l)}(x, \tilde{x}) + 1 \quad (77)$$

Theorem 7 *Suppose that $\phi(x)$ is uniformly Lipschitz.⁸ Then, at initialization,*

$$\nabla_{\theta z_i^{(l)}}(x) \nabla_{\theta z_{i_1}^{(l)}}(\tilde{x}) \rightarrow \delta_{i, i_1} \Theta^{(l+1)}(x, \tilde{x}) \quad (78)$$

where Θ is defined in Definition 4.

⁸That is, $|\phi(x) - \phi(y)| \leq C|x - y|$ for some $C > 0$.

Proof of Theorem 7. The proof is by induction. Recall that

$$\begin{aligned}
x = \text{input} &\in \mathbb{R}^d \\
y^{(l)}(x) &= \begin{cases} x & \text{if } l = 0, \\ \phi(z^{(l-1)}(x)) & \text{if } l > 0. \end{cases} \\
z_i^{(l)}(x) &= \frac{1}{n_l^{1/2}} \sum_j W_{ij}^{(l)} y_j^{(l)}(x) + b_i^{(l)},
\end{aligned} \tag{79}$$

where $y^{(l)}(x), z^{(l-1)}(x) \in \mathbb{R}^{n^{(l)} \times 1}$. We have

$$z_i^{(0)}(x) = \frac{1}{n_0^{1/2}} \sum_{j=1}^{n_0} W_{i,j}^{(0)} x_j + b_j^0 \tag{80}$$

and, hence,

$$\nabla_{\theta} z_i^{(0)}(x) = \left(\frac{1}{n_0^{1/2}} x, 1 \right), \tag{81}$$

so that

$$\nabla_{\theta} z_i^{(0)}(x) \nabla_{\theta} z_{i_1}^{(0)}(\tilde{x})' = \delta_{i,i_1} \left(\frac{1}{n_0} x' \tilde{x} + 1 \right) = \delta_{i,i_1} (\Sigma^{(1)}(x, \tilde{x}) + 1) = \delta_{i,i_1} \Theta^{(1)}(x, \tilde{x}). \tag{82}$$

We now proceed by induction. Suppose we have proved the claim for $z^{(l-1)}$. We have

$$z_i^{(l)}(x) = \frac{1}{n_l^{1/2}} \sum_{j=1}^{n_l} W_{i,j}^{(l)} \phi(z_j^{(l-1)}(x)) + b_i^{(l)} \tag{83}$$

The vector of coefficients of this neural network, θ , can be decomposed into $\theta = (\theta_{l-1}, W^{(l)}, b^{(l)})$, where θ_{l-1} are all the coefficients except for those of the l 'th layer.

Each $z_j^{(l)}(x)$, $j = 1, \dots, n_l$. We have

$$\begin{aligned}\nabla_{\theta_{l-1}} z_i^{(l)}(x) &= \frac{1}{n_l^{1/2}} \sum_{j=1}^{n_l} W_{i,j}^{(l)} \nabla_{\theta_{l-1}} \phi(z_j^{(l-1)}(x)) \\ &= \frac{1}{n_l^{1/2}} \sum_{j=1}^{n_l} W_{i,j}^{(l)} \nabla_{\theta_{l-1}} z_j^{(l-1)}(x) \phi'(z_j^{(l-1)}(x))\end{aligned}\tag{84}$$

and, hence,

$$\begin{aligned}\nabla_{\theta_{l-1}} z_{i_1}^{(l)}(x) \nabla_{\theta_{l-1}} z_{i_2}^{(l)}(\tilde{x})' &= \frac{1}{n_l} \sum_{j_1=1}^{n_l} W_{i_1,j_1}^{(L+1)} \nabla_{\theta_{l-1}} z_{j_1}^{(l-1)}(x) \phi'(z_{j_1}^{(l-1)}(x)) \sum_{j_2=1}^{n_l} W_{i_2,j_2}^{(l)} \nabla_{\theta_{l-1}} z_{j_2}^{(l-1)}(\tilde{x}) \phi'(z_{j_2}^{(l-1)}(\tilde{x})).\end{aligned}\tag{85}$$

By the induction hypothesis, in the limit as $n_{l-1}, \dots, n_1 \rightarrow \infty$, we have

$$\begin{aligned}\nabla_{\theta_{l-1}} z_{i_1}^{(l)}(x) \nabla_{\theta_{l-1}} z_{i_2}^{(l)}(\tilde{x})' &\approx \frac{1}{n_l} \sum_{j_1=1}^{n_l} W_{i_1,j_1}^{(L+1)} \phi'(z_{j_1}^{(l-1)}(x)) \sum_{j_2=1}^{n_l} W_{i_2,j_2}^{(l)} \phi'(z_{j_2}^{(l-1)}(\tilde{x})) \delta_{j_1,j_2} \Theta^{(l)}(x, \tilde{x}) \\ &= \frac{1}{n_l} \sum_{j=1}^{n_l} W_{i_1,j}^{(L+1)} \phi'(z_j^{(l-1)}(x)) W_{i_2,j}^{(l)} \phi'(z_j^{(l-1)}(\tilde{x})) \Theta^{(l)}(x, \tilde{x})\end{aligned}\tag{86}$$

In the limit as $n_l \rightarrow \infty$, we get

$$\begin{aligned}\nabla_{\theta_{l-1}} z_{i_1}^{(l)}(x) \nabla_{\theta_{l-1}} z_{i_2}^{(l)}(\tilde{x})' &\rightarrow \delta_{i_1,i_2} (\sigma_W^{(l)})^2 E[\phi'(z_j^{(l-1)}(x)) \phi'(z_j^{(l-1)}(\tilde{x}))] \Theta^{(l)}(x, \tilde{x})\end{aligned}\tag{87}$$

At the same time,

$$\begin{aligned}
& \nabla_{(W^{(l)}, b^{(l)})} z_{i_1}^{(l)}(x) \nabla_{(W^{(l)}, b^{(l)})} z_{i_2}^{(l)}(\tilde{x})' \\
&= \delta_{i_1, i_2} \left(\frac{1}{n_l} \sum_j \phi(z_j^{(l-1)}(x)) \phi(z_j^{(l-1)}(\tilde{x})) + 1 \right) \\
&\rightarrow \delta_{i_1, i_2} (E[\phi(z_j^{(l-1)}(x)) \phi(z_j^{(l-1)}(\tilde{x}))] + 1) \\
&= \delta_{i_1, i_2} (\Sigma^{(l)}(x, \tilde{x}) + 1)
\end{aligned} \tag{88}$$

□

B. Experimental Setup

This appendix describes the full empirical pipeline. We first present the MLP training configuration (B.1), then detail the construction of PTK and random feature factors (B.2), followed by performance evaluation metrics and statistical diagnostics (B.3), and finally robustness procedures based on multiple random seeds and ensembling (B.4).

B.1. MLP Training Configuration

The MLP models are trained using a rolling-window scheme designed to reflect realistic real-time forecasting and portfolio construction. At each month t , the model is estimated using data from the previous 60 months and then used to generate portfolio weights for month $t+1$. The training procedure, architecture, and optimization details are summarized below.

Model architecture. We employ a standard parametrization MLP (SP-MLP) with ReLU activations. Depths: 1, 2, 4 layers; widths: 16–512 neurons (16, 32, 64, 128, 256, 512). Total configurations: 18 (3×6).

Initialization. Network weights are initialized using the default PyTorch Kaiming uniform scheme, where weights are drawn from $\mathcal{U}(-\sqrt{k}, \sqrt{k})$ with $k = 1/\text{in_features}$, and biases are initialized to zero.

Training scheme. For the initial training window, the model is trained from scratch for 20 epochs. Thereafter, as the training window rolls forward by one month (dropping the oldest observation and adding the newest), the model is warm-started from the previous solution and fine-tuned for 10 epochs. This rolling fine-tuning scheme substantially reduces computational cost while preserving stability and convergence.

Mini-batching and optimization. Optimization is performed using the Adam optimizer, with learning rate $\text{lr} = 2^{-\text{lr_power}}$ and $\text{lr_power} = 16$. Training is conducted with a batch size of one, where each batch corresponds to the entire cross-section of stocks in a given month. Specifically, at each training step, the model simultaneously processes all N_t stocks observed in month t , producing a single portfolio return forecast. This design aligns the training objective directly with the portfolio construction problem.

Loss function and normalization. The model is trained to minimize the maximal Sharpe ratio regression (MSRR) loss. To ensure scale invariance and stable optimization across months with differing cross-sectional sizes, the loss is normalized by the number of stocks in the cross-section. Formally, letting $f_t \in \mathbb{R}^{N_t}$ denote the vector of MLP predictions and $R_{t+1} \in \mathbb{R}^{N_t}$ the realized excess returns, the loss is defined as

$$\mathcal{L}_t = \frac{(1 - f_t^\top R_{t+1})^2}{N_t}.$$

This normalization ensures that each monthly training step contributes comparably to the overall optimization objective, regardless of fluctuations in the cross-sectional sample size.

Training window. Throughout, we employ a rolling training window of length 60 months.

B.2. Factor Construction

B.2.1 Portfolio Tangent Kernel (PTK) Factors

The PTK factors are derived from the gradients of the trained MLP with respect to its parameters. For a given month t with N_t stocks, let $f(X_{i,t}; \theta_t^*)$ denote the MLP prediction for stock i .

- **Gradient Computation:** We compute the gradient of the normalized factor return (MSRR loss) with respect to the network parameters:

$$g_t(\theta_t^*) = \nabla_{\theta} \left(\frac{1}{\sqrt{N_t}} \sum_{i=1}^{N_t} f(X_{i,t}; \theta_t^*) R_{i,t+1} \right) \in \mathbb{R}^{1 \times P},$$

where $R_{i,t+1}$ is the realized excess return and P is the total number of network parameters. Stacking these vectors over time produces the gradient matrix

$$G = \begin{bmatrix} g_1 \\ g_2 \\ \vdots \\ g_T \end{bmatrix} \in \mathbb{R}^{T \times P}.$$

- **Kernel Matrix:** The kernel matrix $\mathbb{K}$ is computed as the uncentered Gram matrix of the gradients:

$$\mathbb{K} = \frac{1}{T}GG' \in \mathbb{R}^{T \times T}.$$

- **Ridge Regression:** We solve for the portfolio weights using ridge regression on the kernel matrix with penalty parameter z .
- **Hyperparameters:**
 - **Ridge penalties** (z_{eff}): 8 values logarithmically spaced from 10^{-5} to 100.
 - **Rolling windows:** $\{12, 60, 120, 240, 360\}$ months.

B.2.2 ReLU (random-feature) Factors

To provide a baseline for the PTK, we generate random-feature factors that approximate infinite-width (dimension $P = 25,000$).

- **Weight Initialization:** Random weights $\Omega \in \mathbb{R}^{P \times d}$ are sampled from a scaled Gaussian distribution:

$$\Omega_{ij} \sim \mathcal{N}\left(0, \frac{2\sigma^2}{d}\right)$$

where d is the input dimension and σ is a scaling parameter.

- **Activations: ReLU:** $\phi(x) = \max(0, x\Omega^T)$.
- **Factor Construction:** The random-feature factors F_t are constructed as man-

aged portfolios, scaled by $N_t^{-1/2}$:

$$F_{j,t} = \frac{1}{\sqrt{N_t}} \sum_{i=1}^{N_t} \phi_j(x_{i,t}) R_{i,t+1}$$

B.3. Random Seeds and Robustness

To ensure the robustness of our results and account for the variability in initialization:

- **Random Seeds:** All experiments are repeated across multiple random seeds (10 seeds).
 - For **MLP** models, the seed controls the initialization of the neural network weights.
 - For **random-feature** models, the seed determines the sampling of the random weight matrix Ω .
- **Ensembling:** In our analysis, we often report the performance of an ensemble of models trained with different seeds. The ensemble return is computed as the volatility-weighted average of the individual seed returns:

$$R_{ensemble,t} = \sum_{k=1}^K w_{k,t} R_{k,t}$$

where $w_{k,t} \propto 1/\widehat{\sigma}_{k,t}$ is the inverse rolling volatility of the k -th seed’s strategy.

B.4. Performance Evaluation and Statistical Tests

We evaluate the strategies using the following metrics:

- **Alpha (α):** The intercept from the OLS regression of the strategy's excess returns (R_{strat}) on the benchmark's excess returns (R_{bench}):

$$R_{strat,t} = \alpha + \beta R_{bench,t} + \epsilon_t ,$$

with Newey–West (1987) standard errors using appropriate lag length.

- **Sharpe Ratio:** The annualized risk-adjusted return, calculated as $\sqrt{12} \cdot \frac{\mu}{\sigma}$, where μ and σ are the mean and standard deviation of OOS monthly returns.

B.4.1 Implicit Regularization and Debiased GRS

To characterize model complexity, overfitting, and generalization behavior, we analyze the spectral properties of the kernel matrix using the following statistics:

- **Implicit Regularization ($Z_*(z)$):** The effective degrees of freedom of the kernel matrix K with ridge penalty z :

$$Z_*(z) = \frac{T}{\text{tr}((zI + K)^{-1})} = \frac{T}{\sum_{i=1}^T \frac{1}{z + \lambda_i}}$$

where λ_i are the eigenvalues of K and T is the window size. We often report the normalized metric divided by $\frac{1}{T}\text{tr}(K)$.

- **Debiased GRS Statistic ($\hat{W}_{\text{debiased}}(z)$):** To correct for the bias in the standard GRS statistic when P/T is large, we compute:

$$\hat{W}_{\text{debiased}}(z)$$

using formula (56).

C. Additional Plots

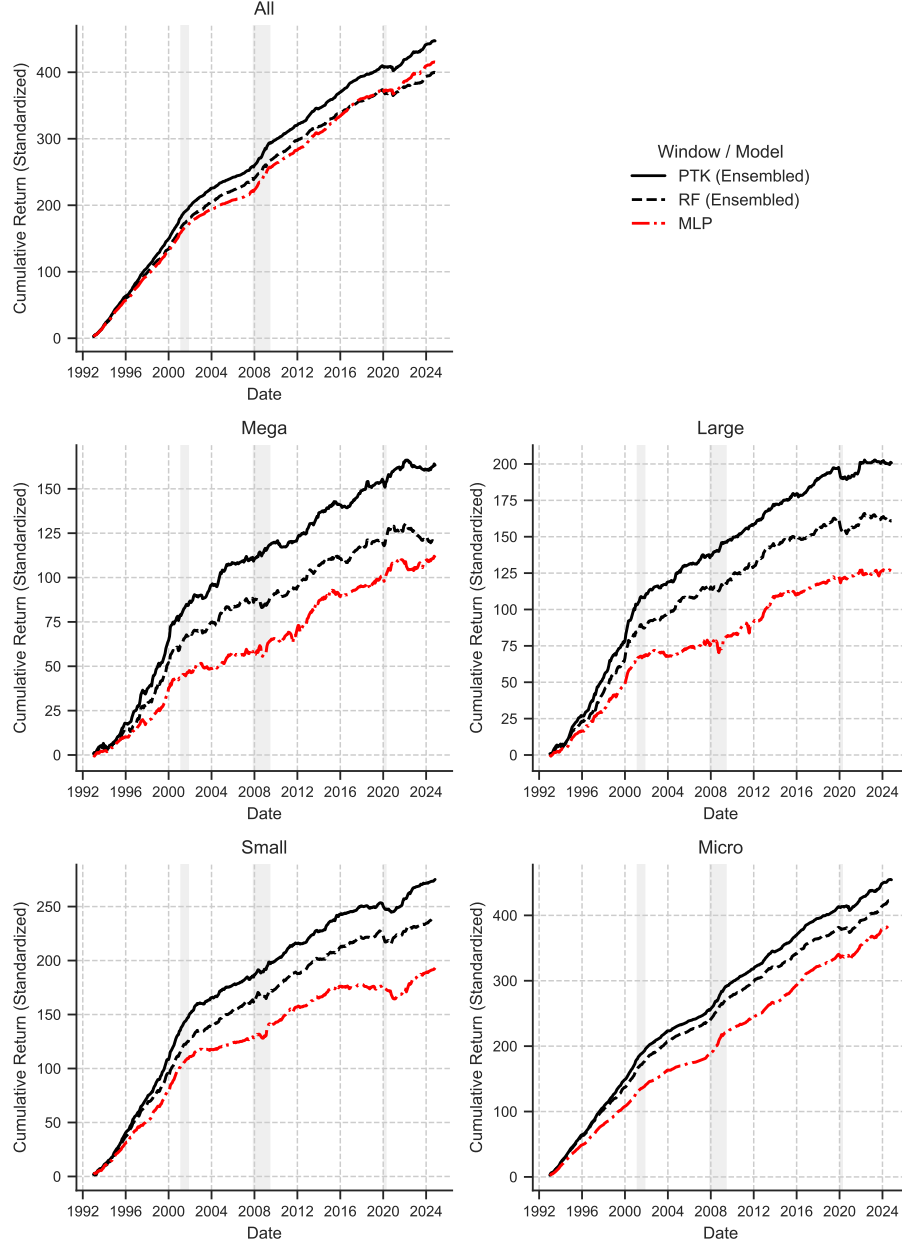


Figure 9: Cumulative Returns of $\bar{R}^{\text{PTK}}$ from (65), $\bar{R}^{\text{ReLU}}$ from (60), and R^{DNN} from (61). We use $D = 2$, $w = 256$ and $z_{\text{eff}} = 10^{-5}$. All returns are normalized to equal full-sample variance.

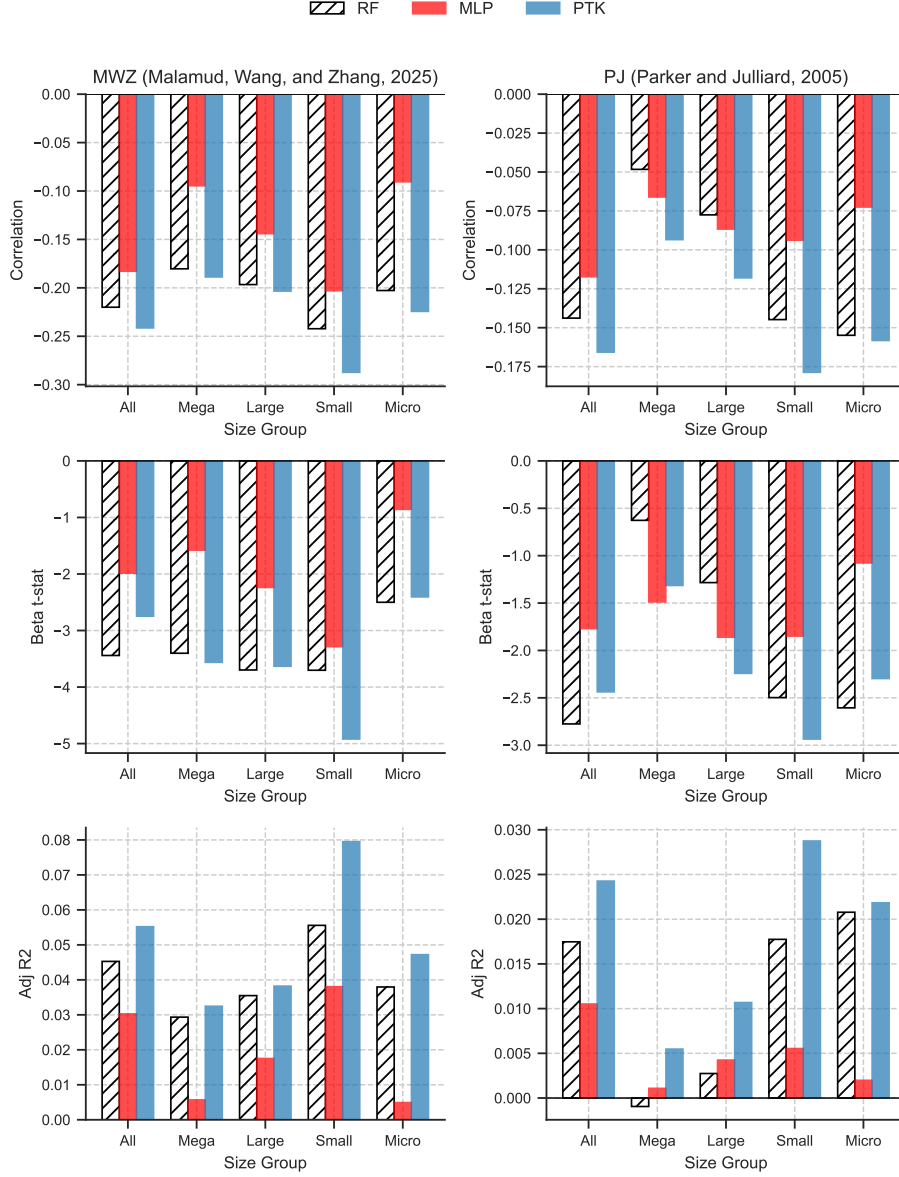


Figure 10: Machine Learning SDFs and Consumption Risk. This figure reports the relation between tradeable machine-learning stochastic discount factors (SDFs) and consumption-based SDFs from [Parker and Julliard \(2005\)](#) (PJ) and [Malamud et al. \(2025\)](#) (MWZ). The top row shows time-series correlations between portfolio returns from the fully trained DNN SDF R^{DNN} (61) (red), the random-feature benchmark $\bar{R}^{\text{ReLU}}$ (60) (grey, striped), and the PTK-based SDF $\bar{R}^{\text{PTK}}$ (65) (blue), and the MWZ future-consumption SDF (left panel) and the PJ long-run consumption SDF (right panel). The middle row reports t -statistics from univariate time-series regressions of each consumption-based SDF on the corresponding tradeable SDF return. The bottom row reports the associated adjusted R^2 . All statistics are shown separately by size group. Consumption-based SDFs are constructed using future realized consumption growth aggregated over horizons of 84 months (MWZ) and 36 months (PJ), respectively, which truncates the sample in June 2018. Ridge regularization with $z_{\text{eff}} = 10^{-5}$ is applied to the ReLU and PTK portfolios.

Swiss Finance Institute

Swiss Finance Institute (SFI) is the national center for fundamental research, doctoral training, knowledge exchange, and continuing education in the fields of banking and finance. SFI's mission is to grow knowledge capital for the Swiss financial marketplace. Created in 2006 as a public-private partnership, SFI is a common initiative of the Swiss finance industry, leading Swiss universities, and the Swiss Confederation.